



에이전트 하네스와 OpenClaw

이건명

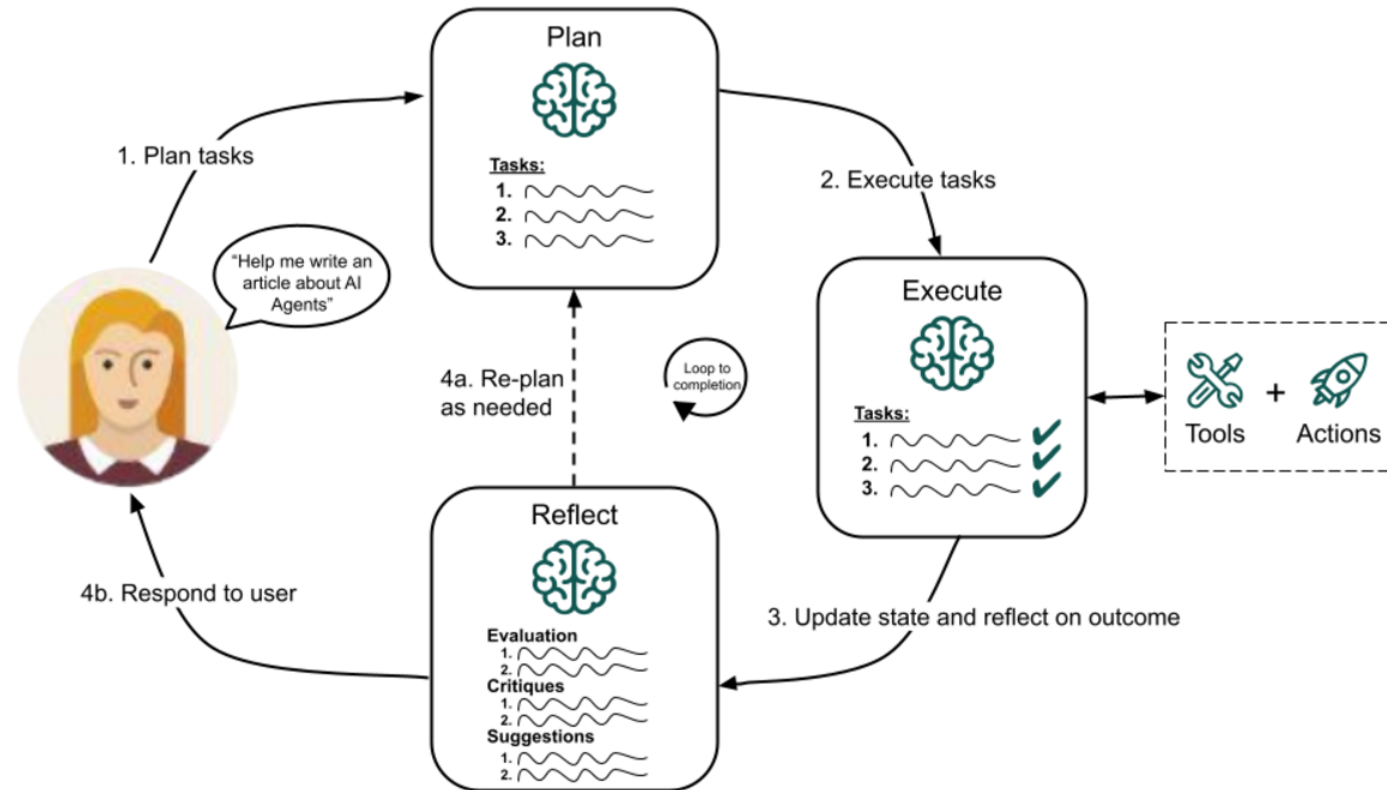
충북대학교 소프트웨어학부

- ❖ 에이전트
- ❖ 에이전트 하네스
- ❖ 컨텍스트 엔지니어링
- ❖ OpenClaw

1. 에이전트의 형태

❖ 에이전트(agent)

- 환경을 **인식**(Perceive)하고, **계획**(Plan)을 세우며, **도구**(Tool)를 활용해 **자율적으로 목표**를 **달성**하는 소프트웨어 시스템



에이전트의 구성요소

❖ 지식기반 및 추론엔진(LLM)

- 자연어 이해, 추론, 생성

❖ 도구 사용(tool calling)

- 웹 검색, 코드 실행, 파일 I/O, API 호출 등 외부 기능을 JSON Schema로 정의해 호출

❖ 메모리 (memory)

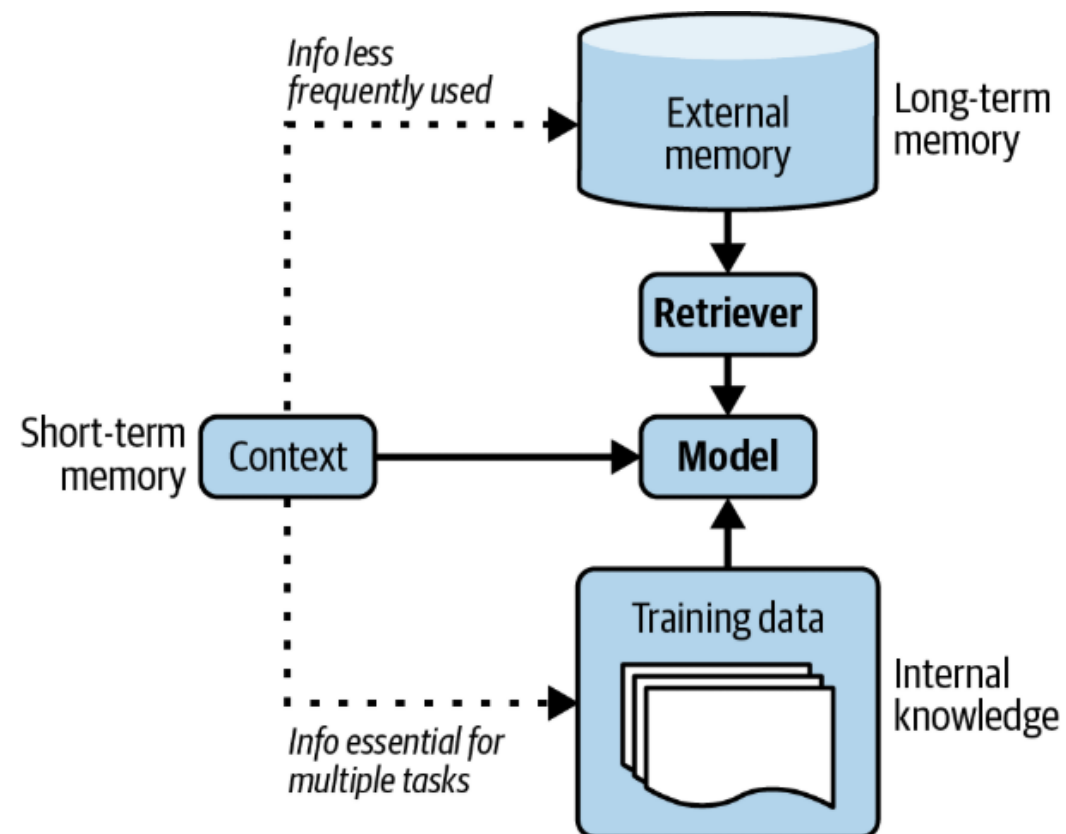
- 에이전트의 과거 입력, 사고, 행동의 이력
- 단기(short-term) 메모리
- 장기(long-term) 메모리

❖ 계획수립(planning)

- 복잡한 목표를 하위 태스크로 분해
- 실행 순서와 의존성 결정

❖ 실행환경(runtime)

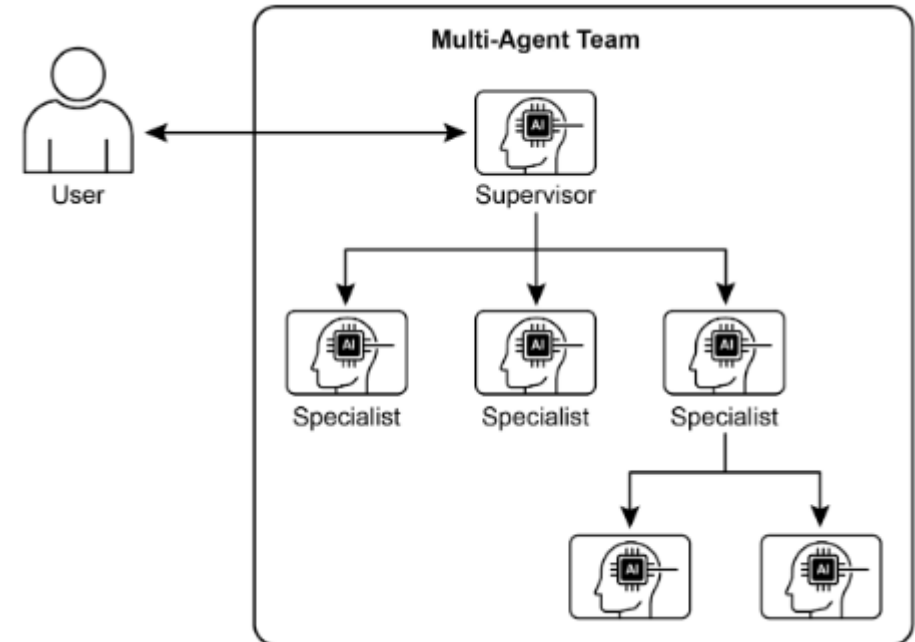
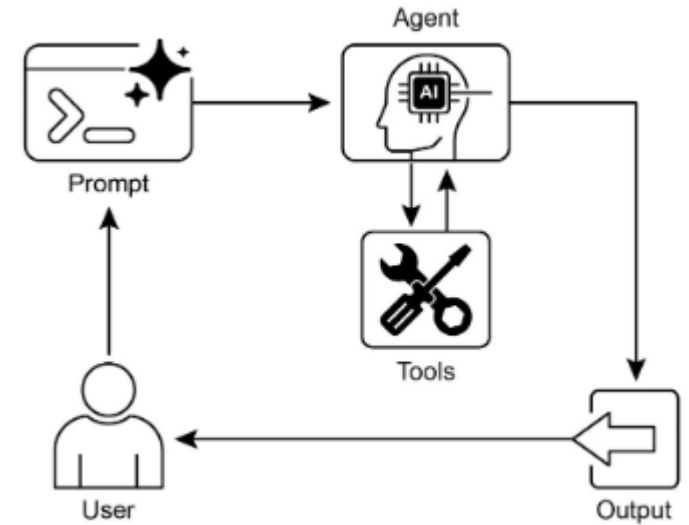
- 에이전트 루프(perceive→plan→act) 관리
- 이벤트 큐, 비동기 실행, 오류 복구 담당



에이전트 실행환경

❖ 에이전트 실행환경의 주요 요소

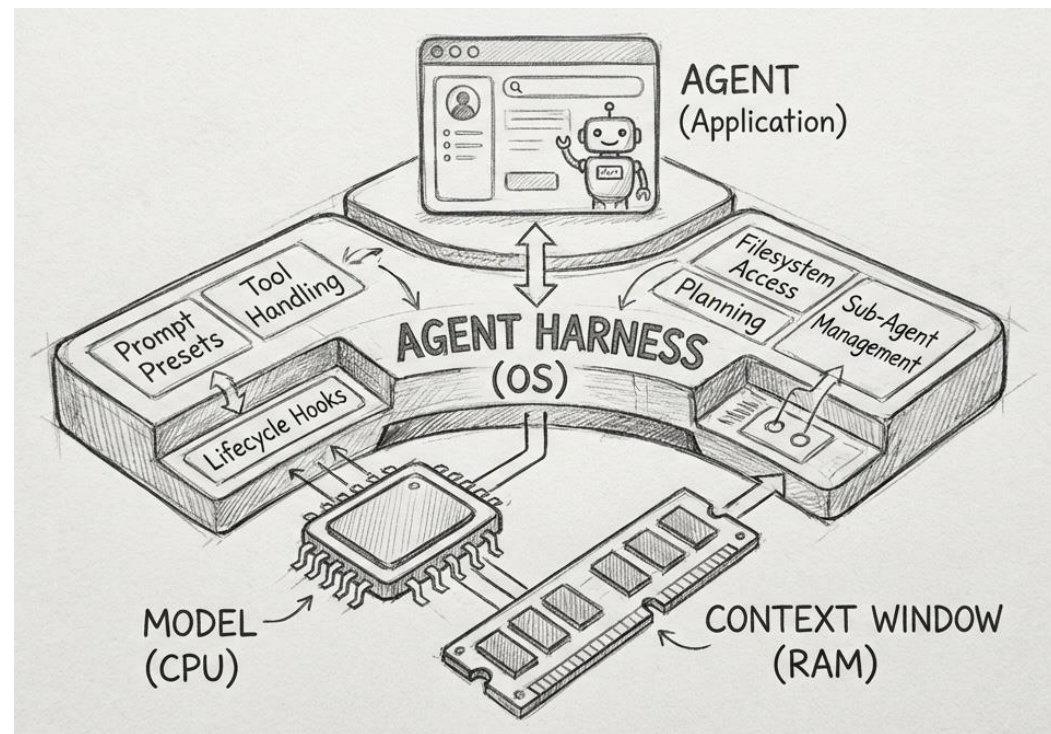
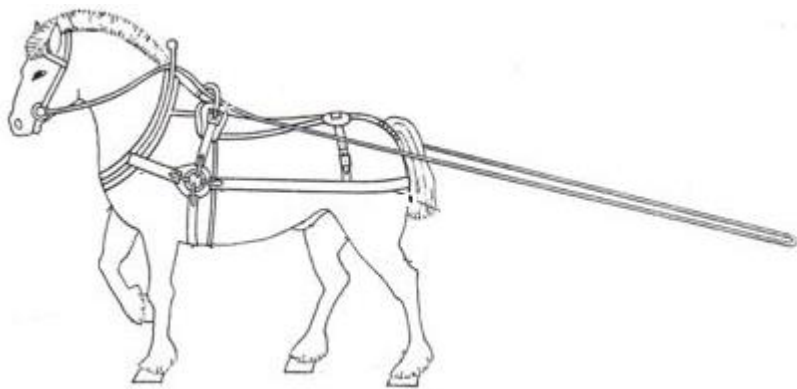
- 이벤트 시스템(event system)
 - event loop, queue system
- 컨텍스트 관리(context management)
- 라우팅(routing)
- 도구 사용(tool use)
 - MCP(Model Context Protocol)
- 능력 확장(capability extension)
 - Skills, hooks
- UI 및 협업
 - 외부 channel
 - 에이전트간 통신: A2A(agent-to-agent), ACP(agent communication protocol)



2. 에이전트 하네스

❖ 에이전트 하네스(harness)

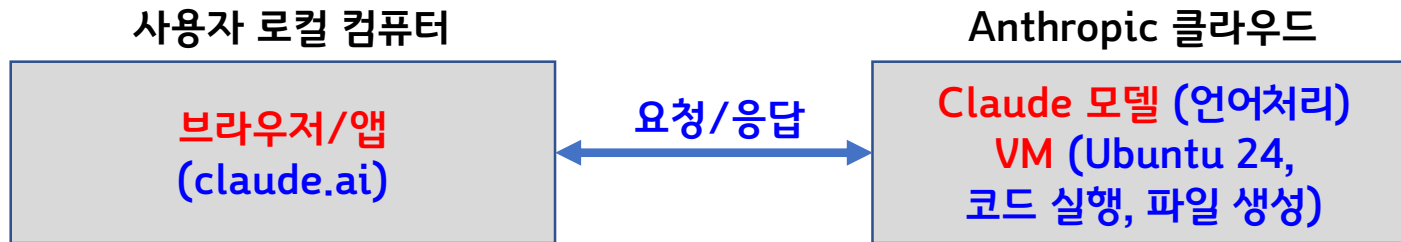
- **에이전트** = **모델(LLM)** + **하네스**
- 프롬프트 사전 설정, 도구 호출에 대한 맞춤형 처리, 라이프사이클 hook 또는 계획, 파일 시스템 접근, 하위 에이전트(subagent) 관리 등 사용할 수 있는 기능 제공
- 예. Claude Code, Codex, ...



Claude Code 하네스

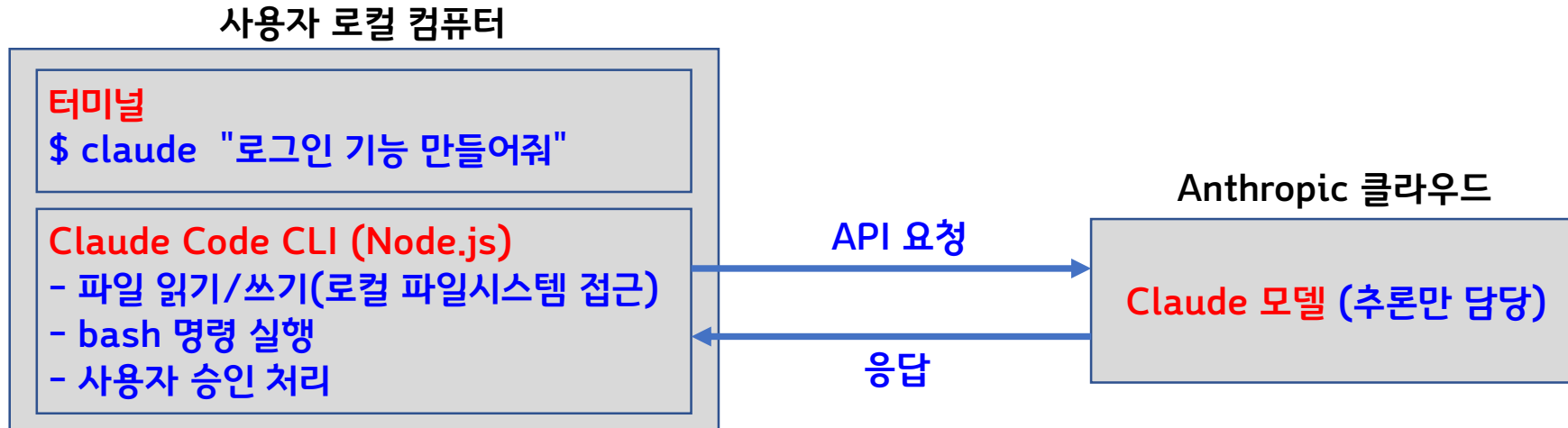
❖ Anthropic 서비스 구조

■ claude.ai 구조



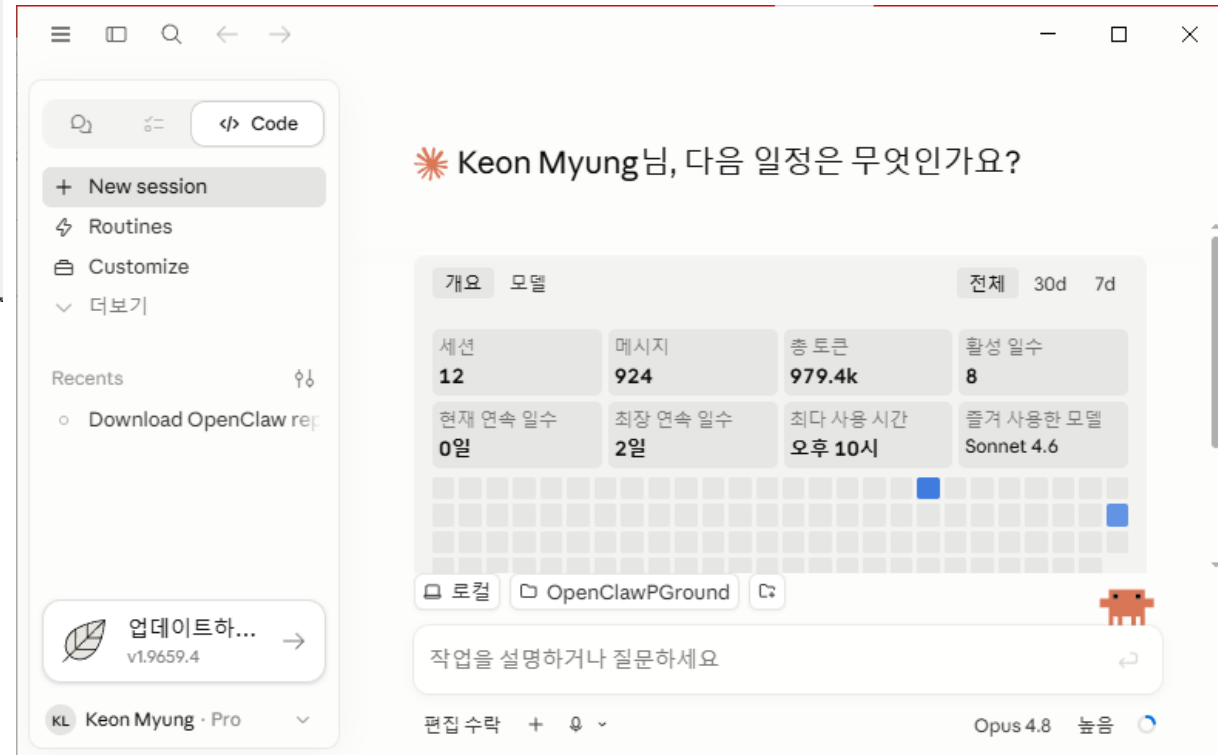
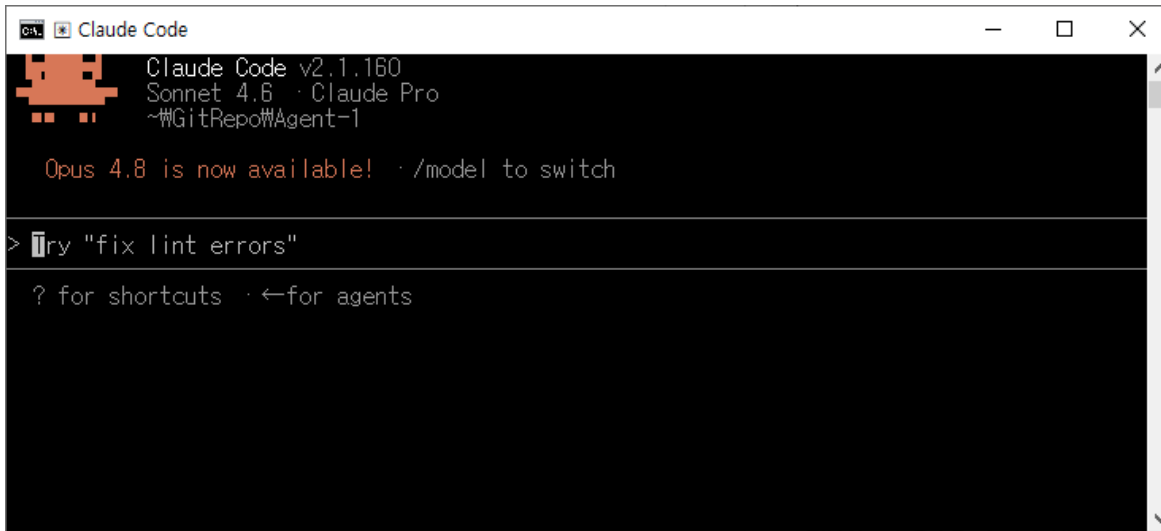
- 각 대화마다 독립된 VM이 할당(격리)
- 대화가 끝나면 VM과 파일시스템 (/home/claude/) 초기화(휘발성)

■ Claude Code



Claude Code 하네스

❖ Claude Code



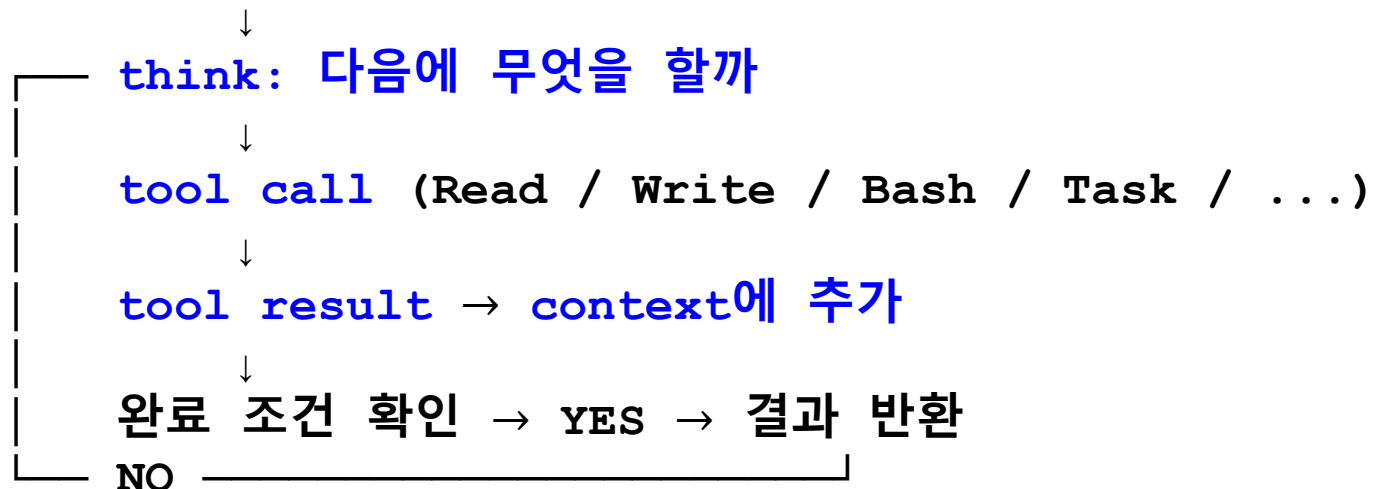
Claude Code 하네스

❖ Claude Code의 에이전트

- agentic loop를 실행하는 Claude 인스턴스
- 단순 질문답변이 아니라 Read → Bash → Write → Bash → ... 같은 반복 루프를 돌며 스스로 목표 달성

Agent 생명주기

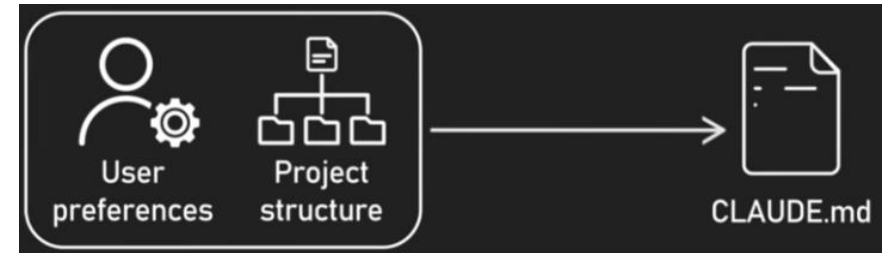
context 초기화



Claude Code 하네스

❖ 에이전트 설정파일 **CLAUDE.md**

- 프로젝트 루트에 위치하여 코딩 규칙과 구조를 Claude에게 메모리처럼 전달
- **/init** 명령어 사용 생성
- Markdown 파일 형식 사용
 - 간결함 유지
 - 가독성
 - 핵심정보 포함
- **Andrej Karpathy**의 CLAUDE.md



The screenshot shows the Claude Code v2.1.76 terminal window. The top bar includes tabs for Problems, Output, Debug Console, Terminal, and Ports. The terminal text reads: 'The default interactive shell is now zsh. To update your account to use zsh, please run 'chsh -s /bin/zsh'. For more details, please visit https://support.apple.com/kb/HT208050. (base) User-MacbookPro-Personal:~ user\$ claude'. Below this, a welcome message 'Welcome back User!' is displayed with a small robot icon. To the right, 'Tips for getting started' and 'Recent activity' are shown. At the bottom, the command '> /init' is entered. The footer shows 'medium · /effort'.

Claude Code 하네스

❖ 실전에서 가장 효과 좋은 작성 패턴(요약)

- **##로 큰원칙을 3~6개로 쪼개고**
 - 큰 주제 단락을 나누는 제목
 - 청킹(chunking) 효과. 우선 순위 힌트. 검색/회상 트리거
- **각 원칙 아래는 -로 체크리스트화**
 - Bullet 1개 = 규칙 1개 (가능하면 한 문장)
- **정말 중요한 금지/필수만 ****bold******
 - 볼드는 “MUST/NEVER”급 3~7개 핵심만
- **“정확히 그대로 써야 하는 것”은 인라인 코드 또는 코드 블록**
 - 따옴표는 “이 표현을 그대로 쓰라”는 뉘앙스
- **따옴표는 리터럴 강제력이 약하니, 정확성은 backticks로 보강**
 - 정확한 리터럴로 취급시키려면 backticks가 더 강력
- **--- (Horizontal rule, 구획 분리)**
 - “여기부터는 결론/검증/부록”처럼 모드 전환 신호
 - 맨 앞부분에 ---과 ---사이에 넣는것 : **front matter**(metadata 포함)

Claude Code 하네스

#CLAUDE.md

Behavioral guidelines to reduce common LLM coding mistakes. Merge with project-specific instructions as needed.

****Tradeoff:**** These guidelines bias toward caution over speed. For trivial tasks, use judgment.

1. Think Before Coding

****Don't assume. Don't hide confusion. Surface tradeoffs.****

Before implementing:

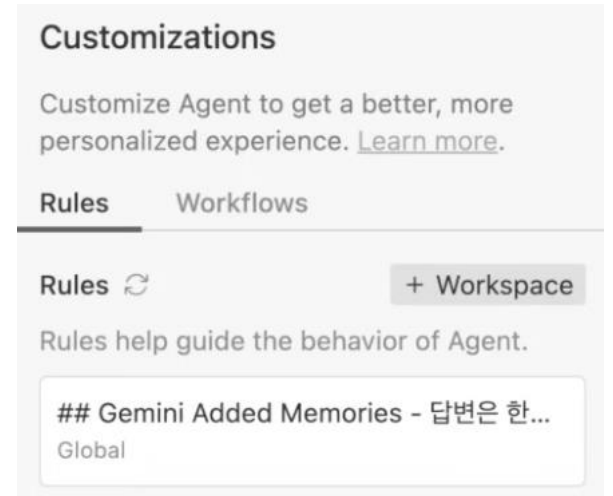
- State your assumptions explicitly. If uncertain, ask.
- If multiple interpretations exist, present them - don't pick silently.
- If a simpler approach exists, say so. Push back when warranted.
- If something is unclear, stop. Name what's confusing. Ask.

2. Simplicity First

****Minimum code that solves the problem. Nothing speculative.****

- No features beyond what was asked.
- No abstractions for single-use code.
- No "flexibility" or "configurability" that wasn't requested.
- No error handling for impossible scenarios.
- If you write 200 lines and it could be 50, rewrite it.

Ask yourself: "Would a senior engineer say this is overcomplicated?" If yes, simplify.



Antigravity

Claude Code 하네스

3. Surgical Changes

****Touch only what you must. Clean up only your own mess.****

When editing existing code:

- Don't "improve" adjacent code, comments, or formatting.
- Don't refactor things that aren't broken.
- Match existing style, even if you'd do it differently.
- If you notice unrelated dead code, mention it -don't delete it.

When your changes create orphans:

- Remove imports/variables/functions that YOUR changes made unused.
- Don't remove pre-existing dead code unless asked.

The test: Every changed line should trace directly to the user's request.

Claude Code 하네스

4. Goal-Driven Execution

****Define success criteria. Loop until verified.****

Transform tasks into verifiable goals:

- "Add validation" → "Write tests for invalid inputs, then make them pass"
- "Fix the bug" → "Write a test that reproduces it, then make it pass"
- "Refactor X" → "Ensure tests pass before and after"

For multi-step tasks, state a brief plan:

```

1.[Step]→verify:[check]

2.[Step]→verify:[check]

3.[Step]→verify:[check]

```

Strong success criteria let you loop independently. Weak criteria("makeitwork") require constant clarification.

**** These guidelines are working if: **** fewer unnecessary changes in diffs, fewer rewrites due to overcomplication, and clarifying questions come before implementation rather than after mistakes.

Claude Code 하네스

❖ Markdown 파일 형식

- 문서의 구조(제목, 목록, 코드, 표 등)를 간단한 기호로 표현하는 경량 마크업 언어
- 텍스트만으로도 읽기 쉬운 원본을 유지하면서, 렌더링하면 보기 좋은 문서(웹/위키/노트/README)
- LLM이 이해하기 쉬운 형태

제목(headings)

큰 제목 (H1)
중간 제목 (H2)
작은 제목 (H3)

문단/줄바꿈

한 줄만 바꾸면 보통 같은 문단
“강제 줄바꿈”은 문장 끝에 공백 2개 또는

굵게/기울임/취소선

****bold****
italic
~~~~strikethrough~~~~

### 목록(불릿 리스트)

- item A
- item B
  - sub item

### 번호 목록(Ordered list)

1. step one
2. step two
3. step three

### 체크리스트(업무 문서에 자주 사용)

- [ ] todo
- [x] done

# Claude Code 하네스

## ❖ Markdown 파일 형식 – cont.

- Markdown 파일 ⇨ 뷰어(viewer)에서 렌더링(rendering) 가능

```
# Knowledge Representation Cheat Sheet

## Knowledge Types
- **Facts (Declarative)**: "Paris is the capital of France."
- **Procedures**: "How to diagnose a problem step by step."
- **Norms/Policies**: "Do not reveal private info."
- **Meta-knowledge**: "How confident we are / what tool to use."

## Example
> If you are unsure, **verify** using a trusted source.

### Mini Table
| Type | Example |
| --- | --- |
| Fact | The exam is on Friday. |
| Procedure | 1) Read 2) Summarize 3) Cite |
```

## Knowledge Representation Cheat Sheet

### Knowledge Types

- **Facts (Declarative)**: "Paris is the capital of France."
- **Procedures**: "How to diagnose a problem step by step."
- **Norms/Policies**: "Do not reveal private info."
- **Meta-knowledge**: "How confident we are / what tool to use."

### Example

If you are unsure, **verify** using a trusted source.

### Mini Table

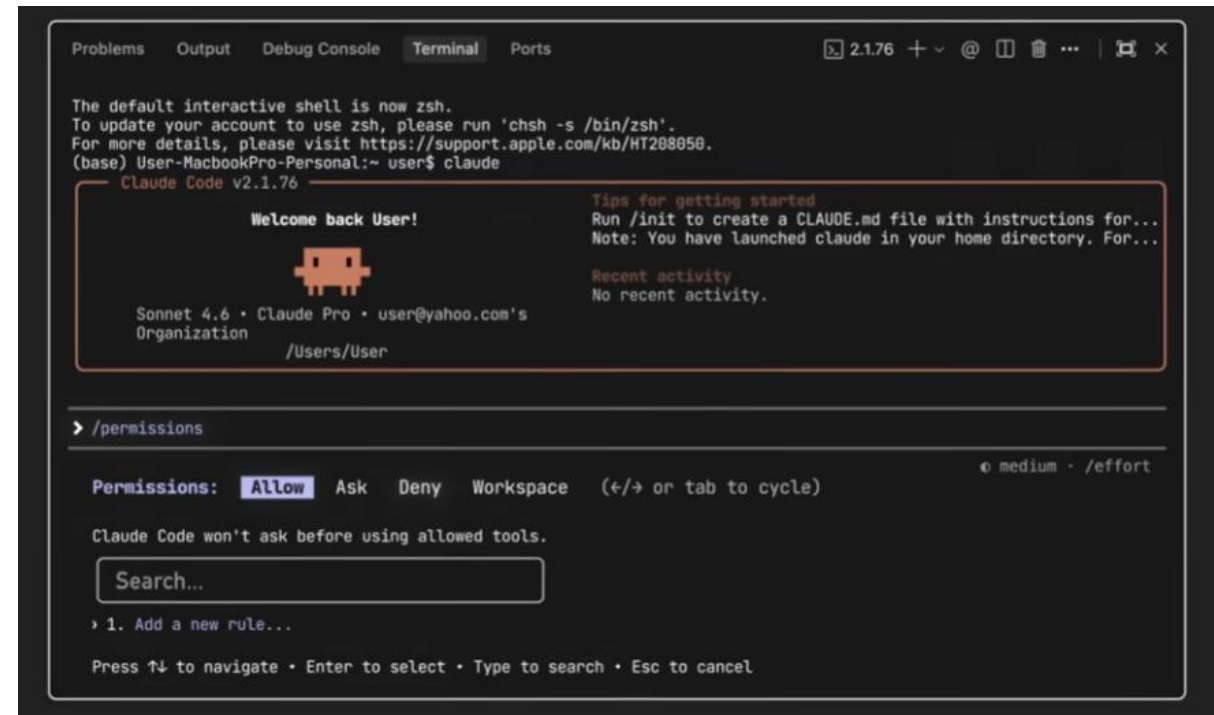
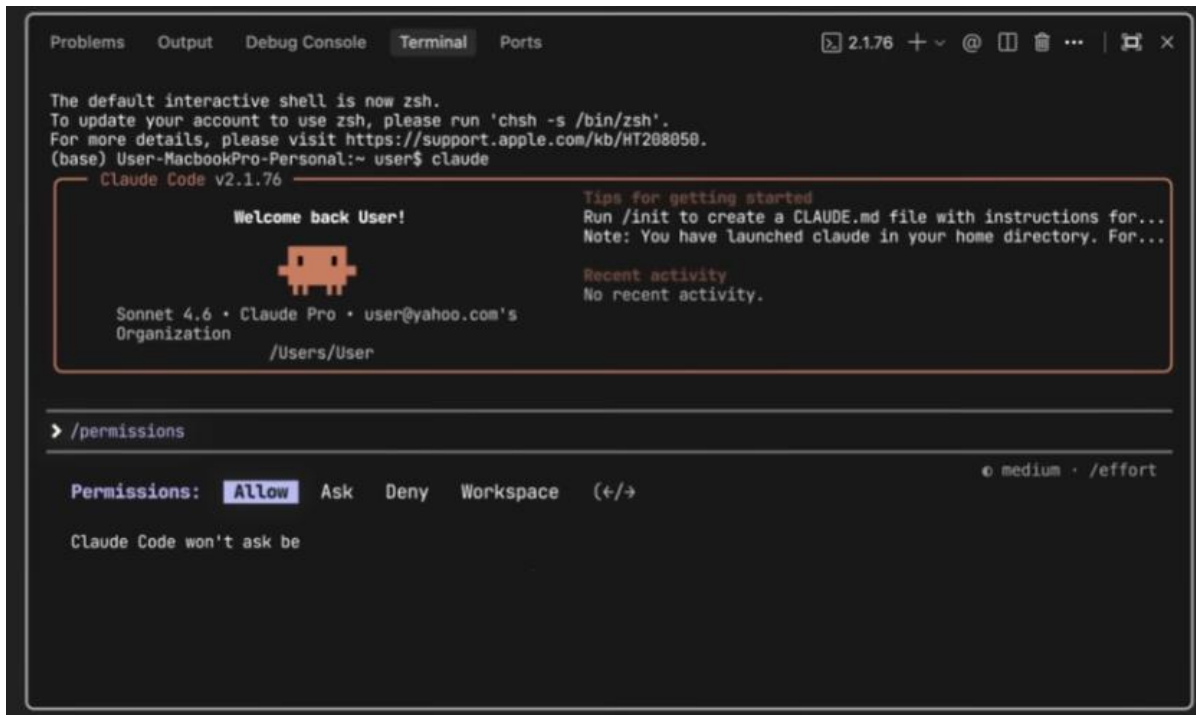
| Type      | Example                      |
|-----------|------------------------------|
| Fact      | The exam is on Friday.       |
| Procedure | 1) Read 2) Summarize 3) Cite |



# Claude Code 하네스

## ❖ AI 권한 설정

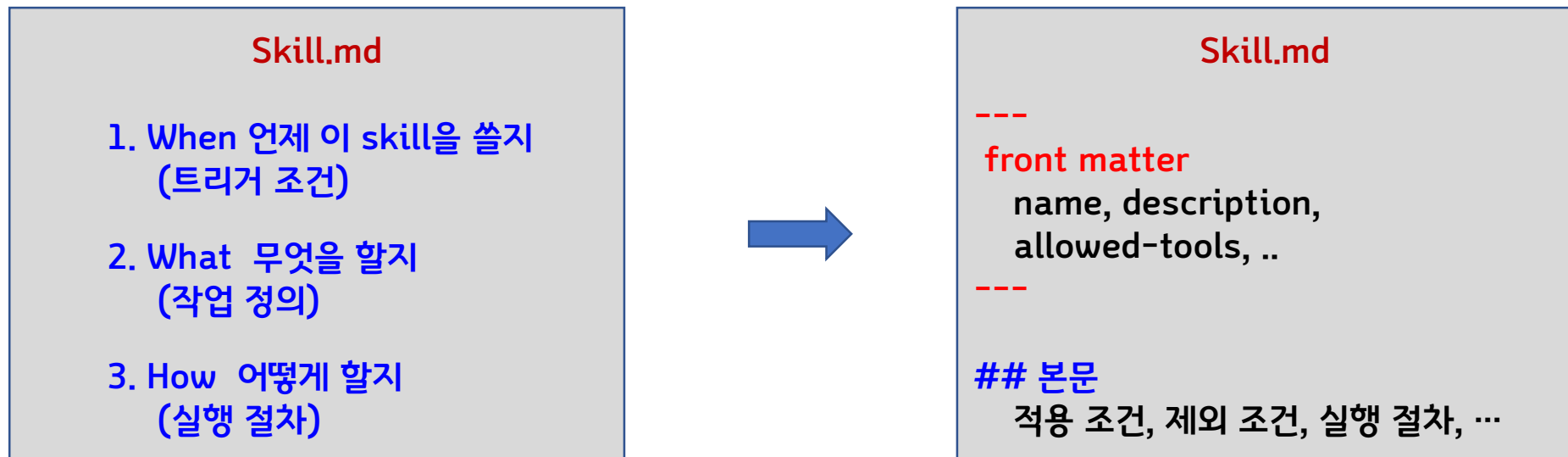
- 코딩 에이전트는 파일 수정이나 Bash 명령 실행 등 강력한 권한
- 세션 중에 **/permissions** 명령어 사용
  - 화면에 나타나는 대화형 메뉴를 통해 원하는 도구를 '허용 목록(allow list)'에 추가하거나 삭제 가능
    - 로컬 파일, 특정 폴더, shell command, GitHub, database, tools 등



# Claude Code 하네스

## ❖ 스킬(Skills)

- 에이전트가 특정 작업을 수행할 때 따르는 전문 지식 패키지
- 특정 작업을 언제(**when**), 무엇을(**what**), 어떻게(**how**) 할지 정의
- **Progressive Disclosure** (점진적 공개) 방식 사용
  - 필요할 때만 적재(load)되어 에이전트의 능력을 확장하는 구조
  - 탐색 시에는 skill.md 파일의 **front matter** 부분만 사용
  - 실제 선택되면 본문 적재하여 사용
- 반복적인 워크플로우를 자동화하기 위한 도구



# Claude Code 하네스

## ❖ SKILL.md 파일의 front matter

- SKILL.md 파일 **최상단의 YAML 블록**
- ---로 감싸진 영역으로, Skill의 **메타데이터**를 정의
- 구성 요소(field)
  - **name** (필수)
    - Skill의 고유 식별자
  - **description** (필수)
    - 트리거(trigger)의 핵심 신호: 스킬을 활성화(load)할지 말지를 결정하는 판단
  - 선택 필드
    - allowed-tools, triggers, version, author, tags, ...

# JSON과 YAML 비교

**JSON:**

```
{  
  "name": "deploy",  
  "version": "1.0.0",  
  "tags": ["db", "infra"]  
}
```

**YAML:**

```
name: deploy  
version: "1.0.0"  
tags:  
  - db  
  - infra
```

# Claude Code 하네스

## ❖ Skills 파일의 front matter 예

```
---
name: deploy                                ← 식별자 (필수)
description:                                 ← 트리거 신호 (필수)
  Use when deploying to production.
  Handles Docker build and ECS update.
allowed-tools: [bash, read]                 ← 허용 도구 (선택)
triggers:                                    ← 명시적 키워드 (선택)
  - "deploy"
  - "배포"
---

## Use this skill when                      ← 적용 조건
- 프로덕션 배포 시
- 스테이징 배포 시

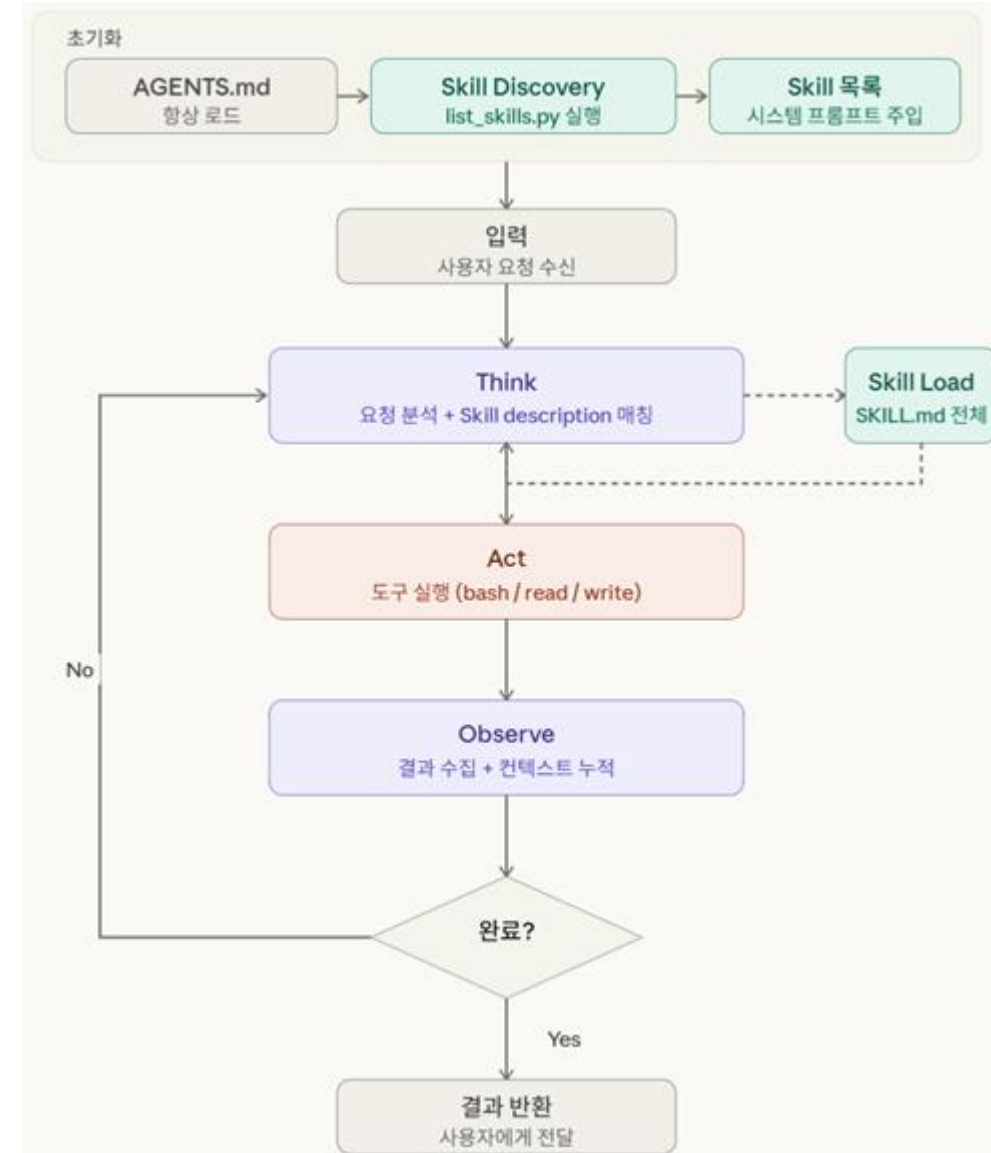
## Do not use when                          ← 제외 조건
- 로컬 개발 환경

## Steps                                    ← 실행 절차
1. 테스트 통과 확인
2. Docker 이미지 빌드
3. 레지스트리 푸시
4. 서비스 업데이트
5. 헬스체크
```

# Claude Code 하네스

## ❖ 에이전트 루프(agent loop)

- 에이전트가 목표를 달성할 때까지 반복하는 실행 사이클
- 목표가 완료될 때까지 **생각→행동→관찰** 반복
  - 초기화: 루프(loop) 시작 전
    - skill 정보 수집 모듈 list\_skills.py 실행  
→ 사용 가능한 skill 목록 파악  
→ **시스템 프롬프트**에 주입 (front matter)
  - 생각(think) 단계: 무엇을 할지 판단
    - 사용자 **요청** vs **Skill description** 매칭  
→ 관련 skill 발견 시 SKILL.md 전체 적재  
→ 실행 방법 파악
    - LLM API 호출 → 결과 → 현재 상태 분석 → 다음 행동 결정
  - 행동(act) 단계: 도구 실행
    - skill의 Steps대로 실행
    - allowed-tools 제한 적용
    - 도구 선택 및 실행
  - 관찰(observe) 단계: **결과 관찰**
    - 결과 도구 실행 결과 수집
    - 성공 / 실패 판단
    - 결과를 컨텍스트(context)에 추가



# Claude Code 하네스

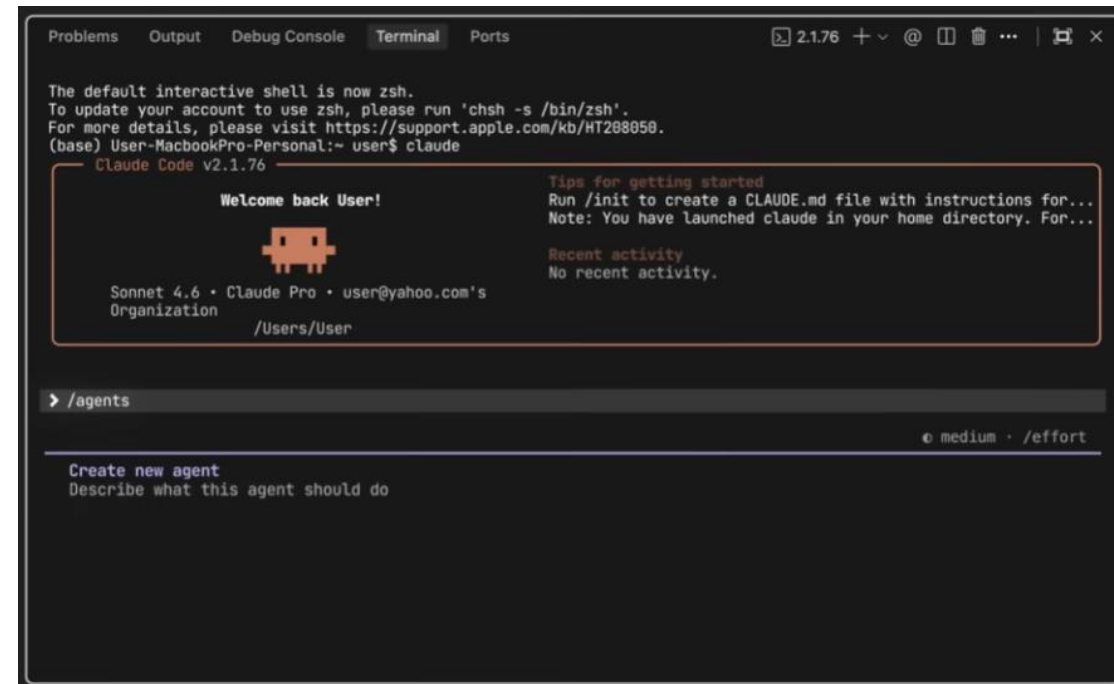
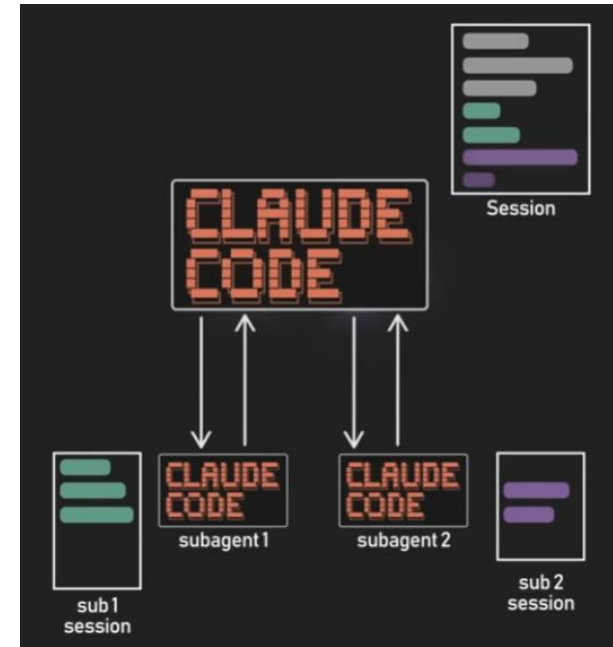
## ❖ 에이전트 루프 – cont.

- **시스템 프롬프트**(system prompt) 구성
  - LLM에게 대화 시작 전에 **역할, 규칙, 컨텍스트**를 부여하는 특별한 지시문
    - 사용자 메시지와 별도로 존재하며, 모델의 동작 방식 전체를 대해 사전에 설정
  - **AGENTS.md** / **CLAUDE.md** 적재(load)
  - list\_skills.py 실행 → **Skill 목록**(name, description) 파악
- 매 루프(Loop)마다 **컨텍스트** 누적
  - Loop 1 [**시스템 프롬프트**] [**사용자 요청**]
  - Loop 2 [**시스템 프롬프트**] [**사용자 요청**] [**행동1**] [**결과1**]
  - Loop 3 [**시스템 프롬프트**] [**사용자 요청**] [**행동1**] [**결과1**] [**행동2**] [**결과2**]
  - ...
  - **대화 이력**(history) 적재 ⇨ Loop가 길어질수록 컨텍스트 증가
  - 토큰 한계 도달 시 오래된 내용 제거
  - 핵심 상태는 별도 저장

# Claude Code 하네스

## ❖ 하위 에이전트(subagent)

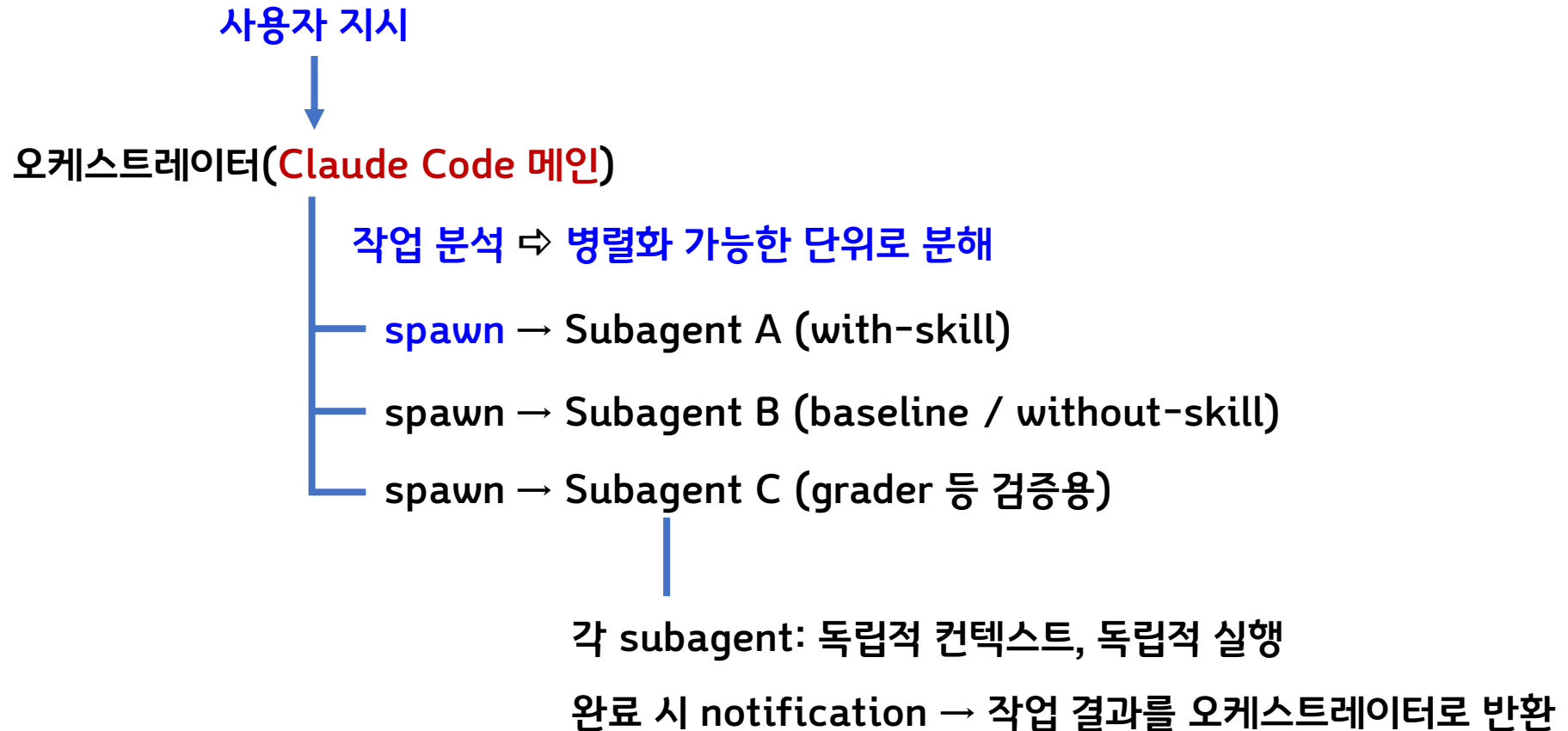
- 복잡한 작업의 분할 수행을 위한 메인 에이전트(세션)과 분리된 에이전트
- **/agents 명령어**
  1. 'Create new agent' 선택
  2. 에이전트에게 이름과 함께 특정 역할을 부여하는 프롬프트 작성
- 독자적인 세션에서 부여받은 작업 수행
- 별도의 컨텍스트 윈도우 관리
- 작업 결과물에 대한 요약정보 ⇒ 메인 세션에 반환



# Claude Code 하네스

## ❖ 하위 에이전트 – cont.

- 사용자가 코딩 지시를 내리면, Claude Code(오케스트레이터)가 작업을 분석해 subagent를 생성

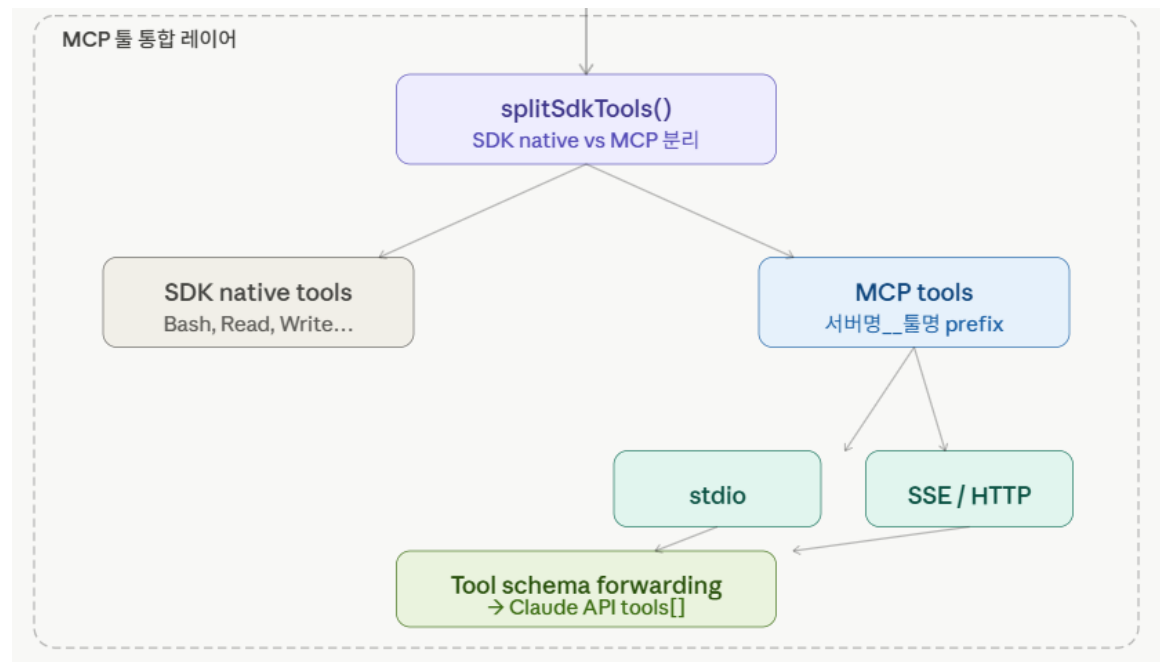
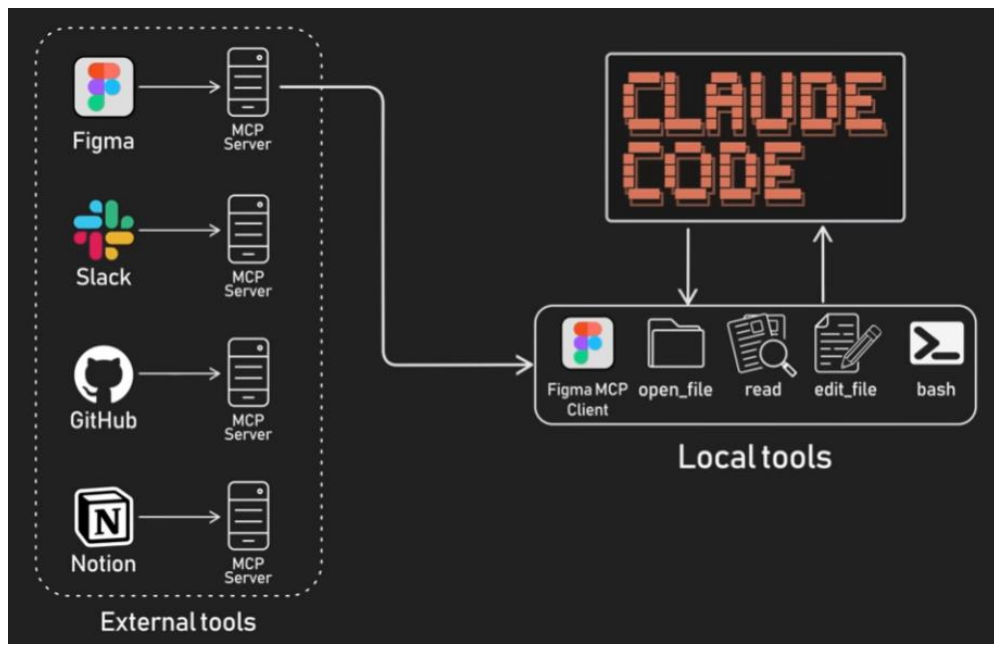




# Claude Code 하네스

## ❖ 외부 도구 연동 MCP (Model Context Protocol)

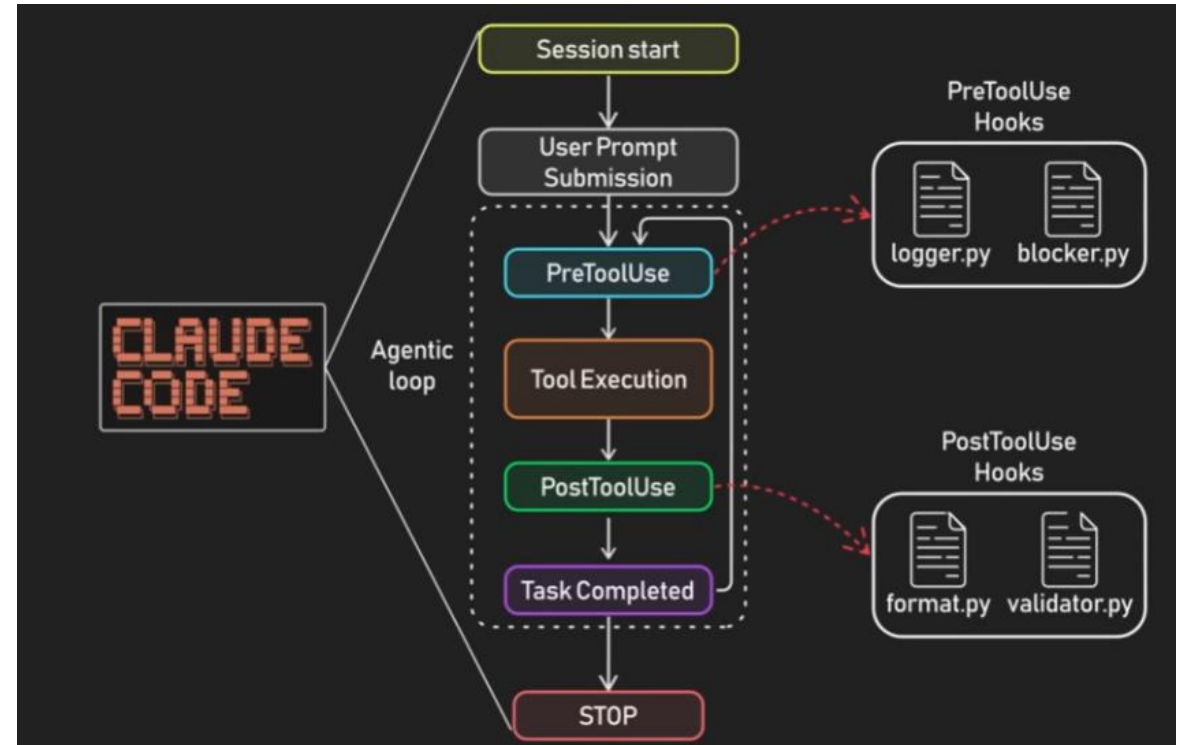
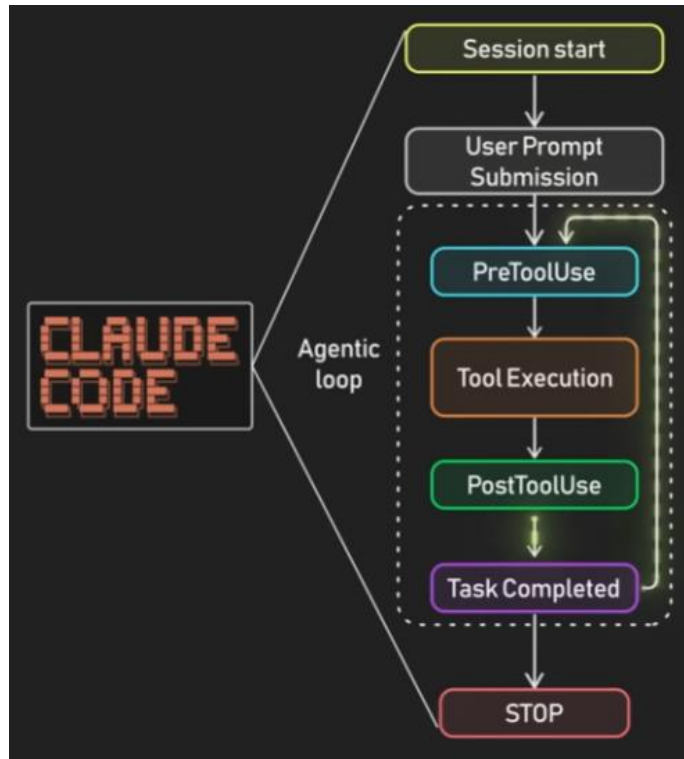
- AI 에이전트가 **외부 도구** 및 데이터와 상호작용할 수 있도록 해주는 **개방형 표준 프로토콜**
- Claude Code는 기본적으로 파일 읽기나 Bash 명령 실행만 가능
- MCP를 활용하면 Figma 파일이나 Slack과 같은 **외부 서비스 및 도구**에 접근
- `claude mcp add my-github -- npx -y @modelcontextprotocol/server-github`
  - ~/.claude/settings.json에 설정정보 저장
- `claude mcp list` # 등록된 서버 목록



# Claude Code 하네스

## ❖ 특정 작업의 자동 반복 수행 Hook

- 특정 상황(도구 실행 전 또는 후)에 매번 수행해야 하는 **확정적인 작업**을 자동으로 실행
  - 코드 포매팅, 보안 검사, 로그 기록
- 일관된 개발 환경과 코드 품질을 유지하기 위한 **자동화 스크립트**



# Claude Code 하네스

## ❖ Hook 구조 및 설정

- Skills로 정의 가능
- 설정 파일: `.claude/settings.json` 또는 `~/.claude/settings.json` (전역)

```
{
  "hooks": {
    "이벤트명": [
      {
        "matcher": "정규식 필터",
        "hooks": [
          {
            "type": "command | http | prompt",
            "command": "실행할 명령"
          }
        ]
      }
    ]
  }
}
```

| 이벤트              | 발화 시점           |
|------------------|-----------------|
| PreToolUse       | 툴 실행 전          |
| PostToolUse      | 툴 실행 후          |
| UserPromptSubmit | 프롬프트 전송 시       |
| SessionStart     | 세션 시작 시         |
| Stop             | Claude가 작업 종료 시 |
| ...              |                 |

- Exit Code 동작
  - exit 0: 성공. stdout에서 JSON 출력 파싱
  - exit 2: 차단 오류

# Claude Code 하네스

## ❖ Hook 구조 및 설정 – cont.

- 예. 위험한 명령 차단 (**PreToolUse**)
  - Bash
    - Linux/macOS의 기본 셸(Shell)
    - 명령어를 실행하고 스크립트를 작성하는 인터페이스

```
# ~/.claude/hooks/block_dangerous.py
import json, sys

data = json.load(sys.stdin)
cmd = data.get("tool_input", {}).get("command", "")
```

```
DANGEROUS = ["rm -rf /", "DROP TABLE", "git push --force", "> /dev/sda"]
```

```
for pattern in DANGEROUS:
    if pattern in cmd:
        print(f"⊗ 차단됨: '{pattern}' 패턴이 감지되었습니다", file=sys.stderr)
        sys.exit(2) # Claude에게 오류로 피드백
```

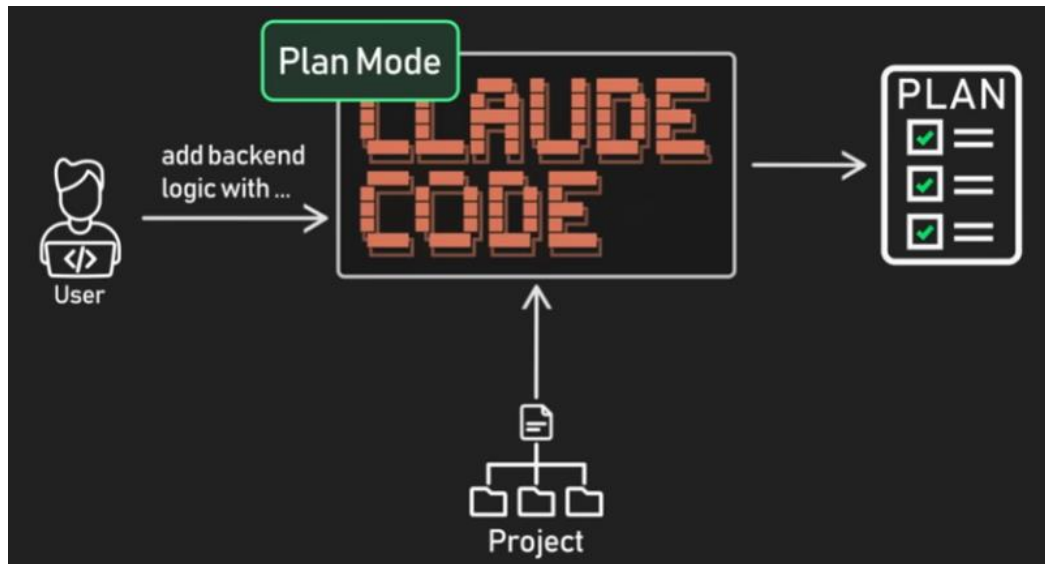
```
sys.exit(0) # 통과
```

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "python3 ~/.claude/hooks/block_dangerous.py"
          }
        ]
      }
    ]
  }
}
```

# Claude Code 하네스

## ❖ 코드 수정 없이 계획만 세우는 **Plan Mode**

- 'Shift + Tab'을 눌러 플랜 모드와 일반 모드 전환 가능
- 플랜 모드에서는 읽기 전용(read-only) 도구만 허용
- Claude가 직접 코드를 수정하거나 변경 불가
- AI가 실제로 코드를 수정하지 않아 토큰 낭비나 잘못된 파일 수정 방지
- 검토 후 실행
  - 사용자가 계획을 검토하고 승인한 후 다시 일반 모드로 전환하면 Claude가 해당 계획에 따라 코드 실행



> 로그인 기능 추가해줘 (plan mode on)

Plan

분석 결과:

- src/components/ 구조 확인
- 현재 auth 관련 파일 없음
- API 엔드포인트: /api/auth 없음

구현 단계:

1. src/components/auth/LoginForm.tsx 생성
  - 이메일·비밀번호 입력 폼
  - 클라이언트 사이드 유효성 검사
  - 성공/실패 상태 처리
2. src/api/auth.ts 생성
  - POST /api/login 엔드포인트
  - JWT 토큰 발급
3. src/hooks/useAuth.ts 생성
  - 로그인 상태 전역 관리
4. App.tsx 수정
  - 로그인 라우트 추가 (/login)
  - 인증 가드 적용

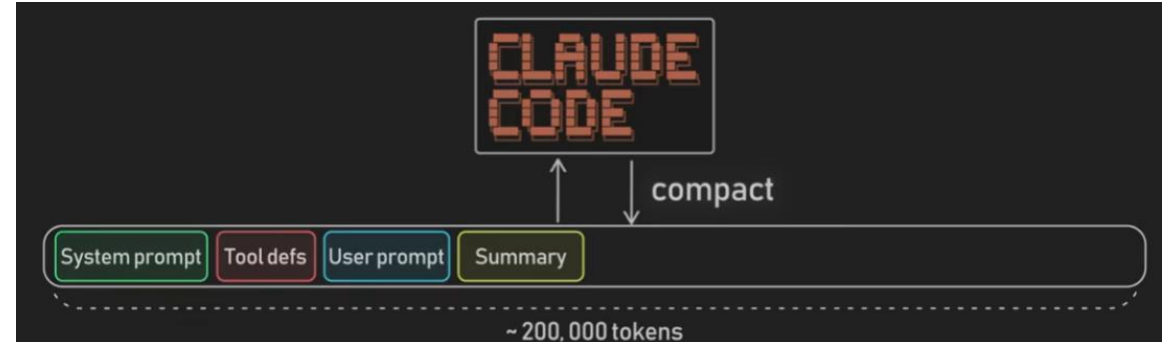
영향받는 파일: 4개 생성, 1개 수정

승인하시겠습니까? (Ctrl+G로 편집, Enter로 실행)

# Claude Code 하네스

## ❖ 컨텍스트 윈도우(context window) 관리

- LLM에 한번에 입력되는 토큰의 전체 개수
- 현재 대화 세션 안에서만 유지
- 현재 상태 확인
  - **/context** 명령어
- 압축
  - **/compact** 명령어
    - 대화 히스토리를 구조화된 요약으로 압축
    - 컨텍스트 윈도우(메모리) 에서 일어나는 일
    - 디스크의 session의 대화기록인 JSONL 파일 그대로 유지



**/compact** 실행 시:

[컨텍스트 윈도우]  
전체 대화 히스토리  
↓ 압축  
요약본으로 대체

[디스크 JSONL]  
~/.claude/projects/  
<session-id>.jsonl  
← 변경 없음 (원본 유지)

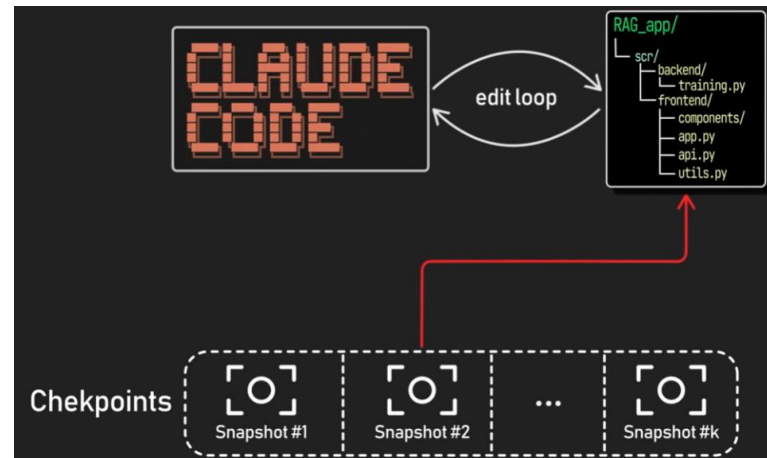
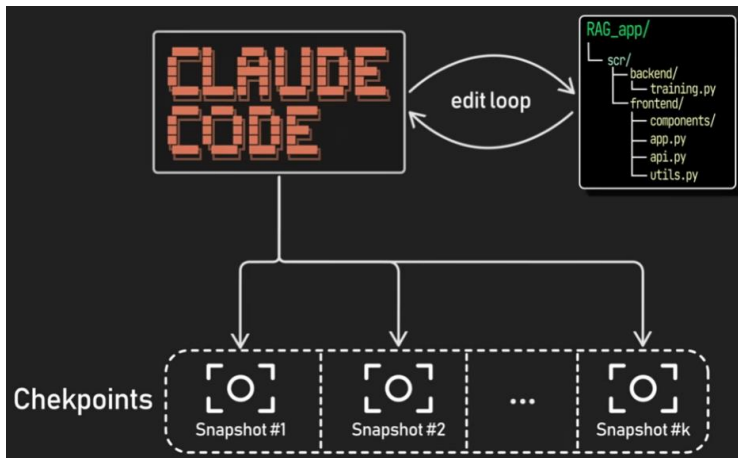
# Claude Code 하네스

## ❖ 코드 잘못 수정시 대처 Checkpoints

- 코드를 수정하기 전, Claude는 **파일의 스냅샷**을 자동으로 생성
- 체크포인트기능 사용 문제가 발생했을 때 손쉽게 **특정 시점으로 복구** 가능
- **/rewind** 명령어 사용
  - rewind 메뉴가 열리면 세션 중 보낸 프롬프트 목록 표시 ⇨ 롤백(rollback) 시점 선택

Restore the code and/or conversation to the point before...

- |                                        |                    |
|----------------------------------------|--------------------|
| <input type="radio"/> "로그인 기능 추가해줘"    | auth.py +45 -0     |
| <input type="radio"/> "버튼 스타일 수정해줘"    | styles.css +12 -3  |
| <input type="radio"/> "API 엔드포인트 만들어줘" | api.py +30 -0 ← 선택 |
| <input type="radio"/> "README 업데이트해줘"  | README.md +10 -2   |



### ✓ 롤백됨

- Claude가 수정한 파일 (Write, Edit, NotebookEdit)
- 대화 컨텍스트
- 토큰 사용 기록

### ✗ 롤백 안 됨

- Bash로 실행한 명령 결과 (pip install, git commit 등)
- 외부 API 호출
- DB 변경사항

# Claude Code 하네스

## ❖ 단일 Claude 인스턴스 기반 복잡한 장기 작업 처리의 문제점

- context window의 작업 중간 소진
- 전문화된 행동들(검색, 코딩, 검증 등)을 하나의 에이전트가 동시 처리 곤란

## ❖ 에이전트 팀(team)

- 복수의 전문화된 에이전트로 팀 구성
- context 격리로 context window 를 분산하고, 역할 전문화로 품질 향상
- 오케스트레이터(orchestrator)와 서브 에이전트(subagent) 구조
  - 오케스트레이터 역할
    - 목표를 받아 서브태스크로 분해하고, 각 서브에이전트에 위임하며, 결과 통합
    - 전체 맥락, 다른 서브에이전트의 상태가 context에 없음
    - 서브에이전트의 raw output 대신 요약을 받아, 자신의 context 보호
  - 서브 에이전트 역할
    - 위임받은 단일 task에 집중
    - 자신의 전문 tool만 가지고 시작
    - 다른 서브에이전트의 존재를 알 필요 없음



# Claude Code 하네스

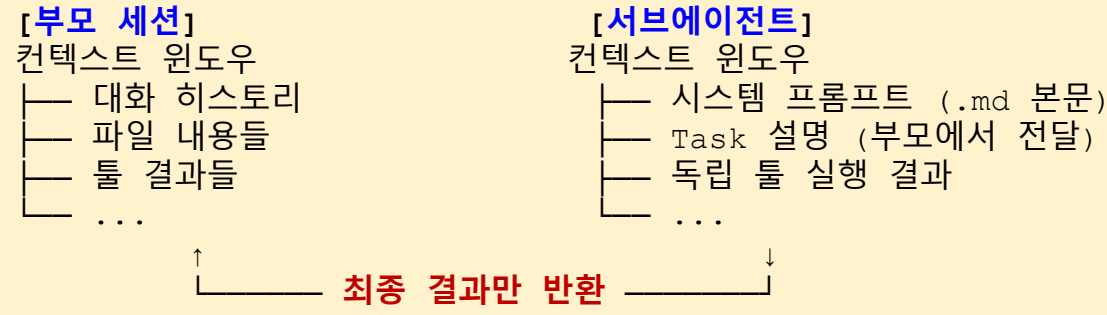
## ❖ 에이전트 팀 – cont.

### ▪ 오케스트레이터 동작

```
class Orchestrator:
    def run(self, goal: str) -> str:
        plan = self.plan(goal)                # 목표 분해

        results = {}
        for task in plan.tasks:
            agent = self.spawn(task)           # 서브에이전트 생성
            raw = agent.execute()              # 독립 context에서 실행
            summary = self.summarize(raw)       # raw → 요약만 수신
            results[task.id] = summary          # orchestrator context 에는 요약만

        return self.integrate(results)
```



# Claude Code 하네스

## ❖ 에이전트 팀 – cont.

- 오케스트레이터 역할 ⇨ **조율**(orchestration)
  - 여러 에이전트를 언제, 어떤 순서로, 어떤 입력으로 실행할지 **스크립트**가 결정하고 관리하는 것
  - 스크립트가 하는 일
    - 에이전트 생성·초기화
    - 태스크 분배 (**어떤 에이전트가 무엇을 할지**)
    - 병렬/순차 실행 제어
    - 중간 결과 전달 (**A의 출력 → B의 입력**)
    - 최종 결과 취합

# Claude Code 하네스

## 서브에이전트 역할 system prompt

### ❖ 에이전트 팀 – cont.

#### ▪ 서브 에이전트 동작

```
RESEARCHER_SYSTEM = """
You are a research specialist. Your only job:
- Search and retrieve information on the given topic
- Evaluate source reliability
- Return a structured summary with citations

Do NOT write code. Do NOT modify files.
Do NOT exceed 3 web searches per query.
"""
```

```
class SubAgent:
    def __init__(self, task: Task, tools: list):
        self.context = {
            "system": f"You are a specialist for: {task.role}",
            "tools": tools,                # 이 task에 필요한 tool만
            "messages": [
                {"role": "user", "content": task.description}
            ]
        }
        # orchestrator context, 타 subagent context = 없음

    def execute(self) -> str:
        return run_agentic_loop(self.context)
```

# Claude Code 하네스

## ❖ Claude Code — agent team 생성

### ▪ Method 1 — **/task** 슬래시 명령 (대화형)

- Claude Code CLI 세션 안에서 **직접 서브에이전트 spawn**
- **현재 Claude 인스턴스**가 **orchestrator**가 되어 서브태스크를 위임
- /task는 현재 대화 흐름을 유지하면서 **병렬적인 서브작업**을 실행할 때 사용
- **orchestrator context**에는 각 /task의 **요약 결과만** 누적

```
$ claude
> /task "src/auth/ 전체를 읽고 SQL injection 취약점 분석 보고서 작성"
# → 새 서브에이전트 인스턴스 생성, 독립 context에서 실행
# → 완료 후 결과 요약은 orchestrator에게 반환

> /task "발견된 취약점 패치 코드 작성 및 테스트"
# → 첫 번째 /task 결과를 input으로 받는 새 서브에이전트
```

1. 부모 에이전트가 Agent 툴로 서브에이전트 호출  
↓
2. 서브에이전트 .md 파일 로드
  - ├── frontmatter → name, description, tools 설정
  - └── 본문 → 시스템 프롬프트로 주입  
↓
3. 새 컨텍스트 윈도우 초기화 (부모와 완전 분리)  
↓
4. 부모가 전달한 Task 내용이 첫 user 메시지로 추가  
↓
5. 서브에이전트 독립 실행  
↓
6. 결과만 부모 컨텍스트로 반환 (중간 과정 미포함)

# Claude Code 하네스

## ❖ Claude Code — agent team 생성

- Method 2 — **Task tool** (에이전트 내부에서 동적 생성)
  - Claude가 스스로 판단해 서브에이전트를 생성하는 방식
  - 에이전트가 agentic loop 도중 "이 부분은 별도 에이전트에게 위임하자"고 결정
  - orchestrator가 서브에이전트에 넘기는 prompt가 곧 그 에이전트의 첫 번째 context가 됨
    - 여기에 필요한 정보를 충분히, 노이즈 없이 담는 것이 핵심

```
# 에이전트의 내부적 동작
Task(
    description="security audit",
    prompt="""
src/auth/ 디렉토리를 읽고 다음을 수행하라:
1. 모든 input validation 함수 식별
2. OWASP Top 10 기준 취약점 검사
3. 발견 사항을 findings.md에 작성

허용 tool: Read, Write, Bash
수정 금지: src/ 외 디렉토리
"""
)
```

# Claude Code 하네스

## ❖ Claude Code — agent team 생성

### ▪ Method 3 — **Headless 파이프라인** (자동화/CI)

- 대화형 세션 없이 CLI 플래그로 에이전트 실행
- 스크립트나 CI/CD 파이프라인에서 에이전트 팀을 조율할 때 사용
- 각 claude 호출은 완전히 독립된 context window로 실행
- 에이전트 간 정보 전달은 파일 시스템을 통해 수행

```
#!/bin/bash
# agent_team.sh - 3단계 에이전트 파이프라인

# Step 1: Researcher agent
claude -p "
docs/api-spec.md를 읽고 미구현 엔드포인트 목록을
todos/unimplemented.md에 작성하라.
" \
--allowedTools "Read,Write" \
--output-format json > researcher_result.json

# Step 2: Coder agent (Step 1 결과를 입력으로)
claude -p "
todos/unimplemented.md의 엔드포인트를 구현하라.
기존 src/api/ 패턴을 따를 것.
각 엔드포인트마다 tests/ 에 테스트 추가.
" \
--allowedTools "Read,Write,Bash" \
--output-format json > coder_result.json

# Step 3: Reviewer agent
claude -p "
방금 구현된 코드를 검토하라:
1. pytest 실행하여 모든 테스트 통과 확인
2. 타입 힌트 누락 확인
3. review_report.md 작성
" \
--allowedTools "Read,Write,Bash" \
--output-format json > reviewer_result.json
```

# Claude Code 하네스

## ❖ Claude Code — agent team

- **CLAUDE.md**로 팀 공통 컨벤션(convention, 관례적인 것) 주입
  - 모든 서브에이전트는 **동일한 CLAUDE.md**를 자동으로 읽음
  - **팀 전체에 적용**되는 규칙 기술

```
## Agent team conventions
- 작업 완료 시 반드시 handoff/ 디렉토리에 요약 파일 작성
- 다른 에이전트가 생성한 파일은 수정 전 반드시 확인
- 에러 발생 시 error_log.md에 기록
- 허가 없이 tests/ 외부 파일 삭제 금지
```

# Claude Code 하네스

## ❖ Claude Code — agent team

- Tool 제한으로 에이전트 역할 강제
  - `--allowedTools` 플래그가 에이전트의 행동 반경을 강력하게 제한
  - 예. Researcher는 Read,Bash(grep),  
Coder는 Read,Write,Bash,  
Reviewer는 Read,Bash처럼  
역할에 맞게 도구를 제한하여 의도치 않은 행동 방지

```
# Researcher: 읽기만 허용
```

```
claude -p "... " --allowedTools "Read,WebSearch"
```

```
# Coder: 쓰기까지 허용, 단 src/만
```

```
claude -p "src/ 이외 디렉토리 수정 금지. ..." --allowedTools "Read,Write,Bash"
```

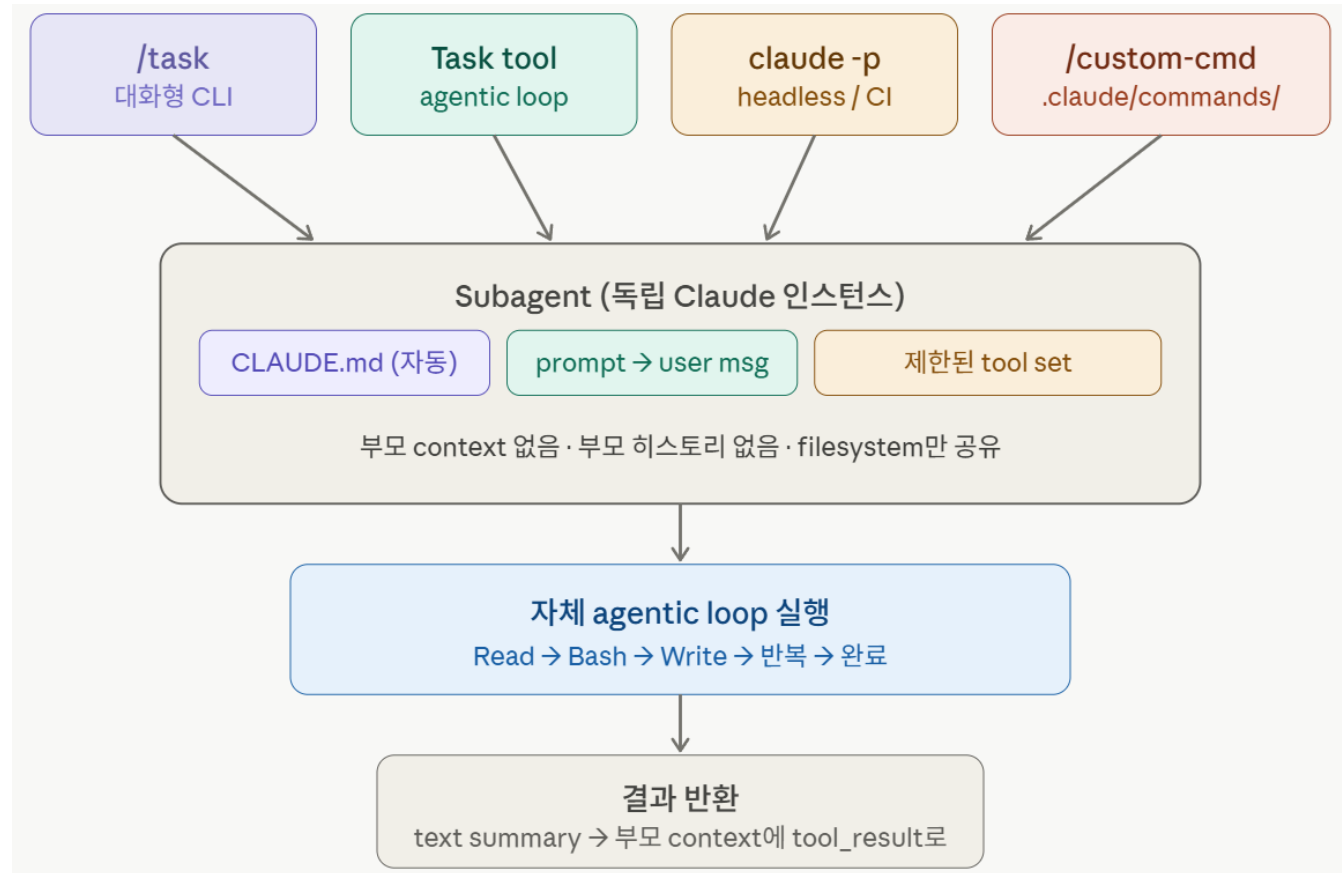
```
# Reviewer: 읽기 + 테스트 실행만
```

```
claude -p "... " --allowedTools "Read,Bash"
```

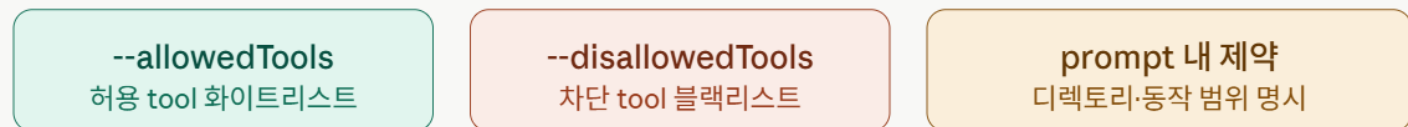


# Claude Code 하네스

## ❖ 서브 에이전트 진입점



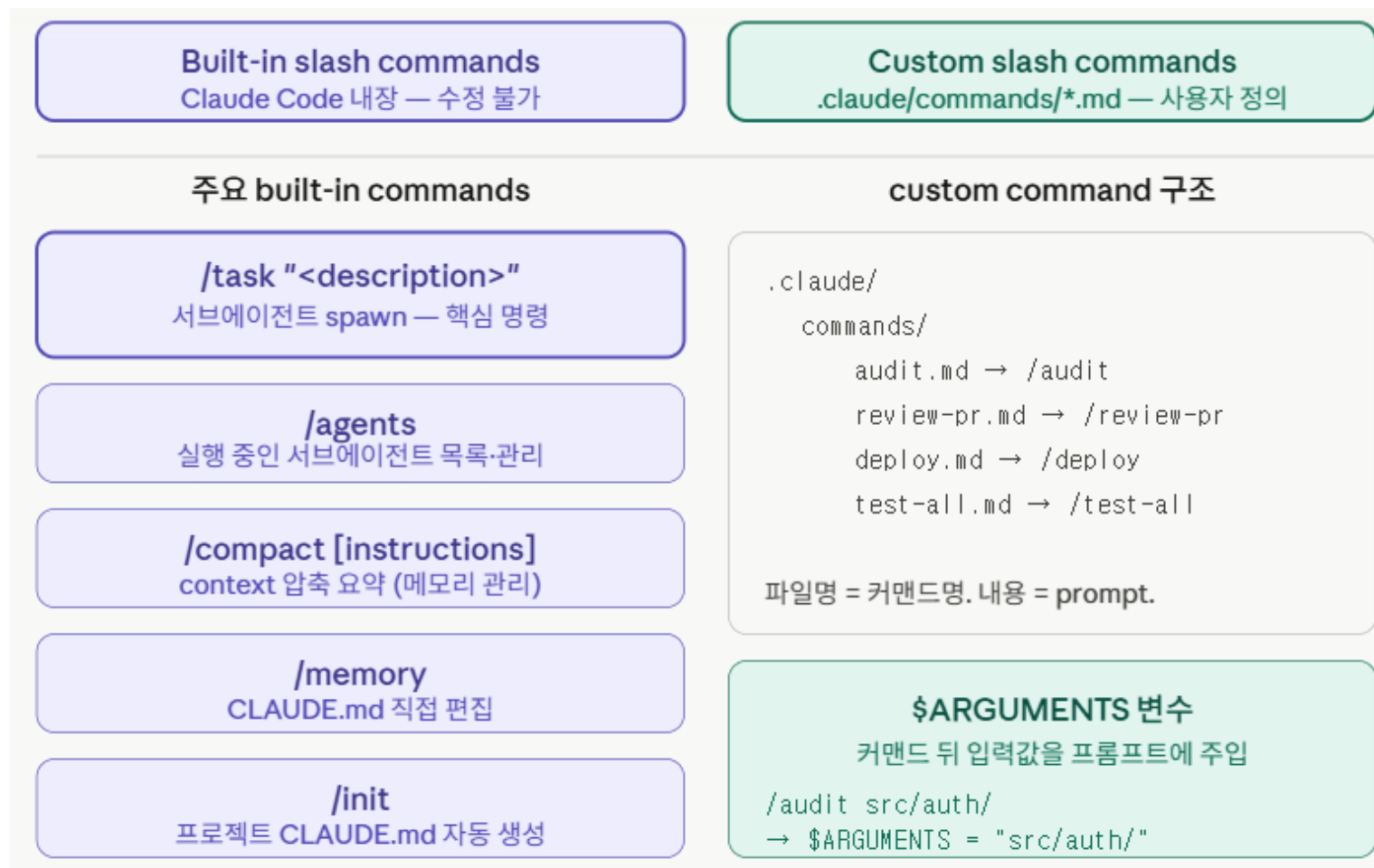
### 서브에이전트 권한 제어



# Claude Code 하네스

## ❖ 슬래시 명령어(slash command)

- 내장 명령어
- 사용자 정의 명령어



# Claude Code 하네스

## ❖ 슬래쉬 명령어 – cont.

- **/init** — 프로젝트 CLAUDE.md 자동 생성

```
> /init
# 코드베이스를 분석해 CLAUDE.md 초안 자동 작성
# 아키텍처, 주요 파일, 빌드 명령 등 자동 추출
```

- **/task** — 서브에이전트 직접 spawn

- 현재 대화 세션이 **orchestrator**, 새 에이전트가 **subagent**
- 현재 대화의 히스토리는 서브에이전트에 전달되지 않음

```
> /task "src/auth/ 전체를 읽고 OWASP Top 10 기준 취약점 분석"
# 새 독립 context의 에이전트가 즉시 실행됨
# 완료 후 결과 요약이 현재 대화에 반환됨
```

- **/agents** — 실행 중인 에이전트 확인

```
> /agents
# 현재 실행 중이거나 완료된 서브에이전트 목록
# 각 에이전트의 상태, task description 확인
```

# Claude Code 하네스

## ❖ 슬래쉬 명령어 – cont.

- **/memory** — CLAUDE.md 실시간 편집

```
> /memory  
# 에디터가 열리며 CLAUDE.md 직접 편집  
# 세션 중 발견한 gotcha (주의사항) 를 즉시 기록
```

- **/compact** — context 압축 (메모리 관리 핵심)

- **Summarization 전략**을 수동으로 트리거하는 명령

```
> /compact  
# 현재 대화 전체를 LLM이 요약  
# 긴 세션에서 window 압박 해소  
# 요약본이 새 context로 대체됨
```

```
> /compact "결정사항과 미완료 작업 위주로 요약"  
# 커스텀 요약 지시 추가 가능
```

# Claude Code 하네스

## ❖ 슬래시 명령어 – cont.

- Custom slash commands — `.claude/commands/`
  - 프로젝트별 반복 작업을 슬래시 커맨드로 패키징
  - 마크다운 파일 하나가 커맨드 하나

```
.claude/  
└─ commands/  
    └─ audit.md          → /audit  
    └─ review-pr.md      → /review-pr  
    └─ test-all.md      → /test-all
```

```
> /audit src/auth/  
> /audit src/api/  
> /audit .          # 전체 프로젝트
```

```
<!-- .claude/commands/audit.md -->
```

\$ARGUMENTS 디렉토리를 보안 감사하라.

## 수행할 것

1. 모든 .py 파일을 읽어 다음을 확인:

- SQL injection 가능성
- 하드코딩된 credentials
- 미검증 user input
- 취약한 crypto 사용

2. 발견 사항을 security/audit\_report.md에 작성:

- CRITICAL / HIGH / MEDIUM / LOW 분류
- 각 발견 항목에 파일명·줄 번호 포함

3. CRITICAL 항목이 있으면 즉시 요약 출력

## 제약

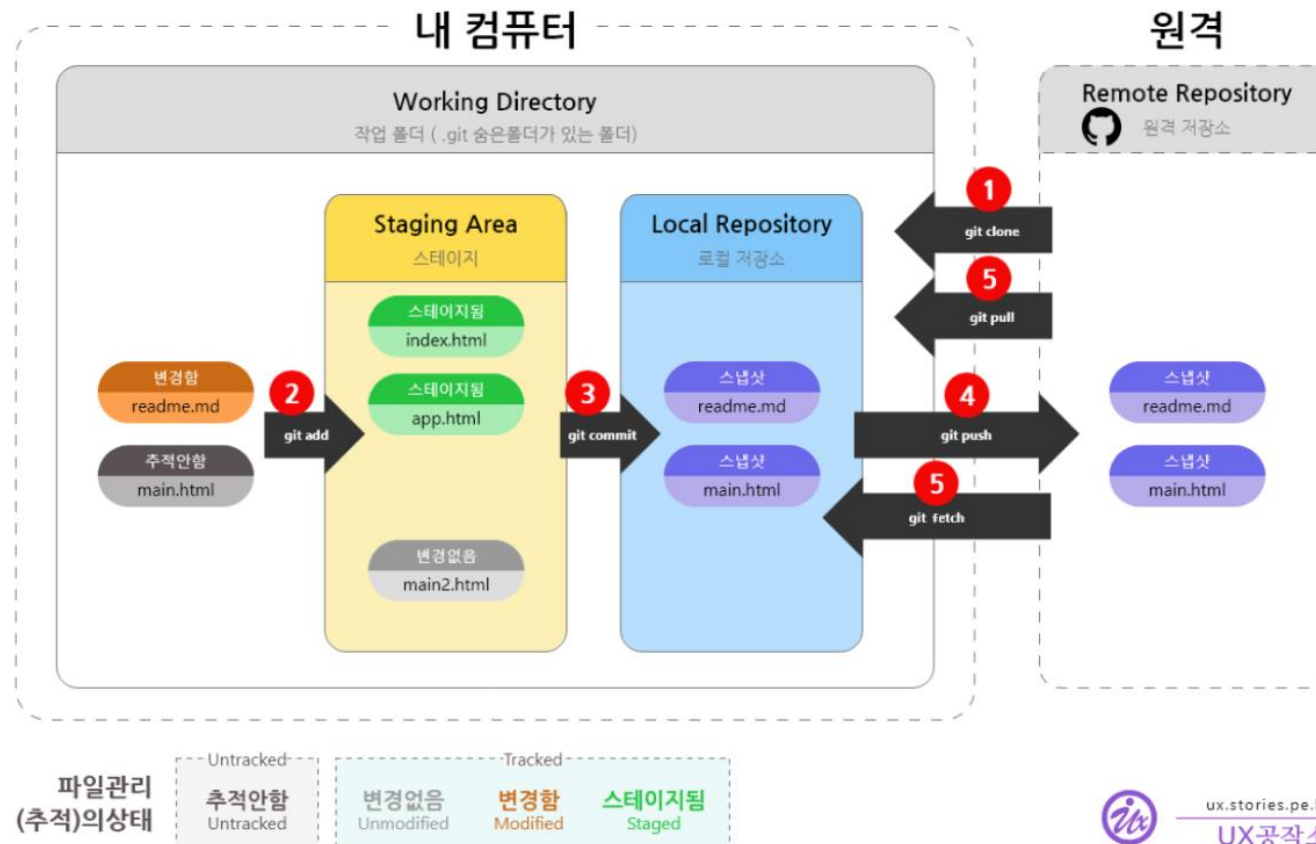
- 코드 수정 금지 (분석만)
- tests/ 디렉토리 제외

# Claude Code 하네스

## ❖ Github 연결

### ■ Git

- 파일변화를 시간에 따라 기록하여 나중에 특정시점의 버전을 다시 꺼내올 수 있는 **버전 관리 시스템**
- 이전 상태 복귀, 시간에 따른 비교, 이슈 추적, 여러 작업자의 병렬 작업



# Claude Code 하네스

## ❖ Github 연결 – cont.

### ▪ Git – cont.

- **git init**
  - 현재 디렉터리를 Git 저장소로 초기화
  - local repository에서 **.git/ 폴더** 생성
- **git add .**
  - 현재 디렉터리의 모든 변경 파일을 스테이징 영역에 추가
  - 각 파일의 blob 객체가 **.git/objects/**에 생성
- **git commit -m "feat: initial commit"**
  - 스테이징된 내용을 로컬 저장소에 영구 저장
  - tree 객체와 commit 객체가 **.git/objects/**에 생성
  - 1. 스냅샷 생성: 현재 파일 상태를 blob·tree로 저장
  - 2. 메타데이터 기록: 작성자, 시간, 메시지
  - 3. 체인 연결: parent 해시로 이전 커밋과 연결
- **git diff HEAD~1 HEAD**
  - 두 커밋 간 차이는 tree를 비교해 계산 (저장된 것은 아님)

```
.git/
├── HEAD           ← 현재 브랜치 (ref: refs/heads/main)
├── config         ← 저장소 설정
├── description
├── hooks/        ← 훅 스크립트 자리
├── info/
├── objects/      ← 비어 있음 (커밋 전)
│   ├── info/
│   └── pack/
├── refs/
│   ├── heads/   ← 비어 있음 (커밋 전)
│   └── tags/
```

### Commit후 추가되는 것

```
.git/
├── objects/
│   ├── ce/013625... ← blob (파일 내용)
│   ├── 4b/825dc6... ← tree (디렉터리 구조)
│   └── 9f/a8c3b1... ← commit 객체
├── refs/
│   └── heads/
│       └── main      ← 커밋 해시 저장됨 (최초 커밋 후 생성)
```

# Claude Code 하네스

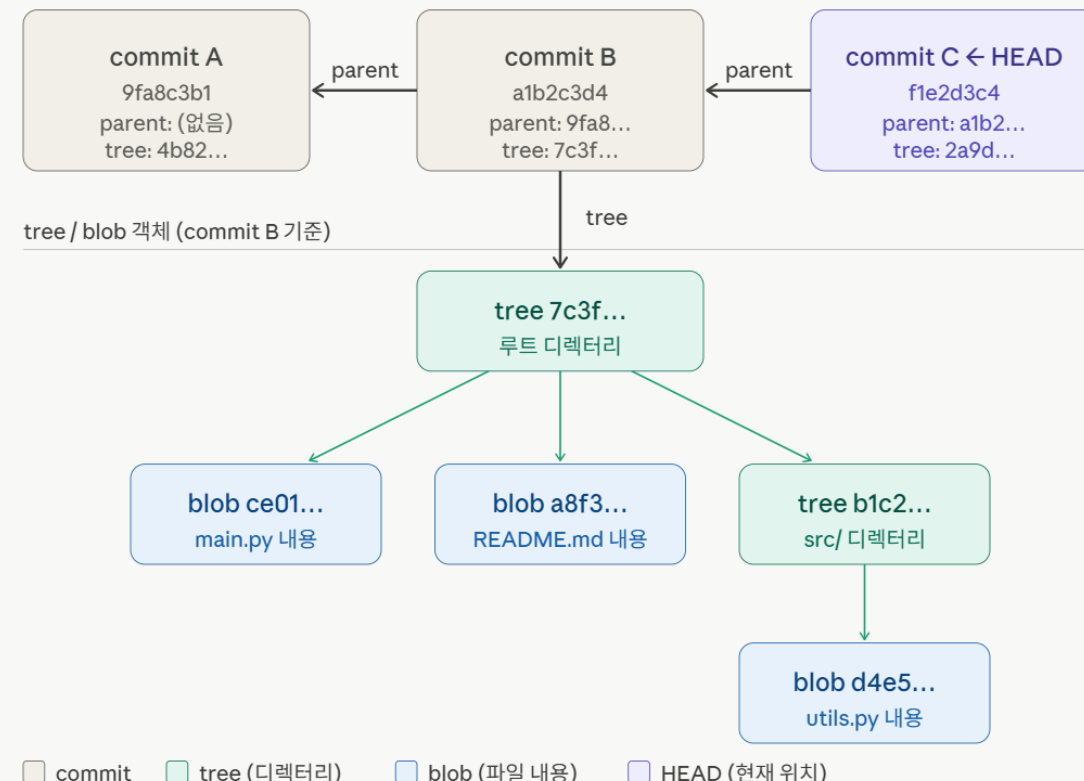
## ❖ Github 연결 – cont.

- 커밋 체인(commit chain)과 스냅샷(snapshot)
  - local repository
    - .git/objects 에 저장
    - 각 파일의 내용을 해싱(hashing)된 코드를 폴더명(2개 문자)와 파일명(나머지문자들)로 사용
    - 수정된 파일에 대해 별도 해싱 및 파일 관리
    - 중복된 파일은 이전 파일 내용 그대로 사용
  - 각 커밋은 변경사항(diff)이 아니라 **그 시점 전체 파일 시스템의 스냅샷**을 가리킴
  - 변경되지 않은 파일의 blob은 이전 커밋과 **동일한 해시를 재사용**하므로 **중복 저장 없음**
  - 커밋 = 스냅샷(blob+tree) + 메타데이터 + parent 포인터
- git push -u origin main
  - 로컬 저장소 main에서 원격 저장소 origin으로 전송

C:\Users\kml\GitRepos\Agent-1\git\objects 디렉터리

```
2026-06-01 오후 01:53 <DIR> .
2026-06-01 오후 01:53 <DIR> ..
2026-06-01 오후 12:58 <DIR> 20
2026-06-01 오후 12:58 <DIR> 2d
2026-06-01 오후 12:58 <DIR> 4c
2026-06-01 오후 12:28 <DIR> 68
2026-06-01 오후 12:59 <DIR> 6f
2026-06-01 오후 12:58 <DIR> 70
2026-06-01 오후 12:59 <DIR> 77
2026-06-01 오후 12:59 <DIR> 78
2026-06-01 오후 12:28 <DIR> 7f
2026-06-01 오후 12:28 <DIR> a4
2026-06-01 오후 01:00 <DIR> b3
2026-06-01 오후 12:28 <DIR> df
2026-06-01 오후 12:28 <DIR> info
2026-06-01 오후 12:28 <DIR> pack
0개 파일 0 바이트
16개 디렉터리 178,779,009,024 바이트 남음
```

커밋 체인





# Claude Code 하네스

## ❖ Github 연결 – cont.

### ■ Github

- Git 저장소를 **클라우드에서 호스팅**하는 플랫폼
- 코드 저장·협업·배포 자동화의 중심

#### Repository (저장소)

- └─ 코드 + 전체 Git 히스토리 저장
- └─ Public (공개) / Private (비공개)
- └─ Fork – 타인 저장소를 내 계정으로 복사

#### Branch

- └─ main ← 기본 브랜치 (배포용)
- └─ develop ← 개발용
- └─ feature/\* ← 기능 개발용

#### Pull Request (PR)

- └─ 브랜치 → main 병합 요청
- └─ 코드 리뷰 공간
- └─ 승인 후 merge

#### Issue

- └─ 버그 신고 / 기능 요청
- └─ 작업 단위로 관리
- └─ PR과 연결 가능 (#이슈번호)

마지막 3개 커밋 보여줘

```
Bash(git log --oneline -3)
└─ 3e72699 feat: add the function for tool use output
   b3c830c feat: initial update
   ✚ 68cb729 Initial commit
```

●마지막 3개 커밋입니다:

| 해시      | 메시지                                        |
|---------|--------------------------------------------|
| 3e72699 | feat: add the function for tool use output |
| b3c830c | feat: initial update                       |
| 68cb729 | Initial commit                             |

# Claude Code 하네스

## ❖ Github 연결 – cont.

- **PR**(pull request)
  - 브랜치(branch) 병합(merge) 요청
    - 저장된 브랜치를 main에 합쳐달라는 요청  
`gh pr create --title "feat: add login"`
  - 저장된 브랜치를 main에 합쳐달라는 요청
  - cf. push
    - 로컬 커밋을 원격 저장소에 전송
  - PR은 원격에 올라간 브랜치를 기반으로 생성
  - push는 PR의 전제 조건

# 3. Context Engineering

## ❖ 컨텍스트 엔지니어링

- 유한한 **토큰 예산**(token budget)의 효과적인 관리
- "얼마나 넣을 수 있나"의 문제가 아니라, **무엇을 넣고 무엇을 버릴지** 전략적으로 결정해야 하는 자원 배분 문제

[시스템 프롬프트] + [메모리] + [도구정보] + [세션 이력] + [검색된 문서] + [사용자 프롬프트]



≤ **N 토큰**(제한 사항)

- 컨텍스트 윈도우의 예산적 관점
  - **희소성**
    - 토큰은 유한 → 모든 정보를 넣을 수 없음
  - **기회비용**
    - A를 넣으면 B를 못 넣음
  - **비용**
    - 입력 토큰 = 직접적인 API 비용
  - **품질 트레이드오프(tradeoff)**
    - 불필요한 정보가 많으면 attention이 분산 → 성능 저하 (lost-in-the-middle 문제)

# Context Engineering

## ❖ 컨텍스트 엔지니어링 전략

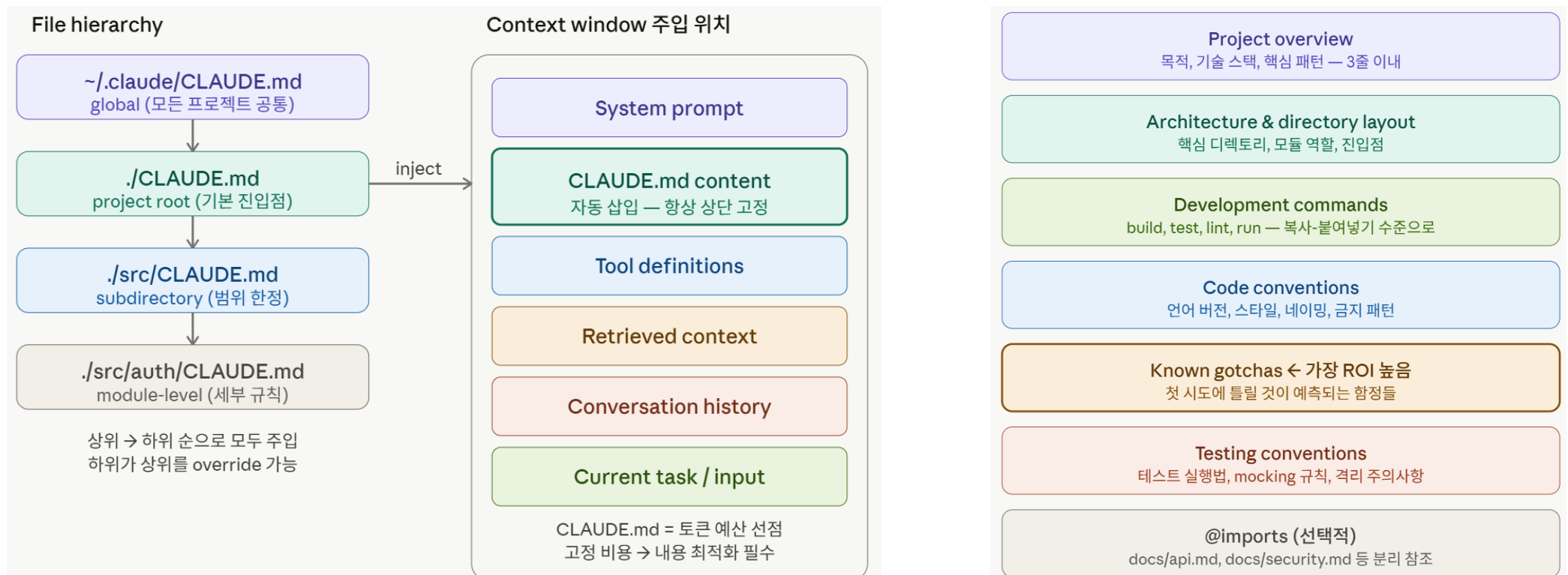
- **압축**(Compression)
  - 대화 히스토리 ⇨ 요약으로 대체
  - 긴 문서 ⇨ 관련 청크만 추출
- **제거**(Eviction)
  - 오래된 메시지 슬라이딩 윈도우 방식으로 제거
  - 중요도 기반 우선순위 제거
- **지연 로드**(Lazy Loading)
  - 모든 도구정의(tool definition)를 처음부터 넣지 않고 필요할 때만 적재
- **예산기반 검색**(Budget-aware Retrieval)
  - RAG에서 "상위 K개" 고정치 아니라 남은 토큰 예산에 맞게 동적으로 청크 수 조절

|                      |         |            |
|----------------------|---------|------------|
| System Prompt        | ~5-15%  | (고정)       |
| Tool Definitions     | ~5-10%  | (선택적 로드)   |
| Long-term Memory     | ~10-20% | (압축/요약)    |
| Retrieved Context    | ~20-40% | (동적 선택)    |
| Conversation History | ~10-20% | (슬라이딩 윈도우) |
| Current Input        | ~5-10%  | (현재 요청)    |
| Output Reserve       | ~10-20% | (응답 공간)    |

# Context Engineering

## ❖ Claude code에서 CLAUDE.md 역할

- Claude Code에서 **context window**에 **자동 주입**되는 프로젝트 수준의 지식 파일
- 에이전트가 코드베이스를 처음 볼 때부터 "**이미 온보딩된 상태**"로 만들어주는 역할
- 에이전트의 **장기 기억**을 context window에 미리 박아두는 역할
- 매번 retrieval하지 않아도 되는 **고빈도·고확실성 지식**을 항상 window 상단에 고정



# Context Engineering

## ❖ Claude code에서 **CLAUDE.md 역할** – cont.

- Context Engineering 관점의 설계 원칙
  - **토큰 예산**(Token budget) **의식**
    - CLAUDE.md는 **매 턴(turn)마다 context window 차지**
    - 길면 길수록 retrieved context, conversation history에 쓸 공간 축소
    - 목표는 **500-1500 tokens 이내**로 압축
  - **계층적 분리** (Hierarchical scoping)
    - 전체 프로젝트 **공통 규칙**은 **root**에, **모듈별 세부 규칙**은 **해당 디렉토리 CLAUDE.md**에 포함
    - 에이전트가 src/auth/에서 작업할 때는 root + src + src/auth 세 파일이 모두 주입
  - **명령형 서술** (Imperative writing)
    - 설명문이 아니라 **지시문**으로 사용
    - # 나쁜 예
      - » 이 프로젝트는 Python 3.11을 사용하고 있습니다.
    - # 좋은 예
      - » Python 3.11 사용하라. 로그 메시지에 f-문자열 절대 금지하라.
  - **@import로 분할 참조**
    - 파일이 커지면 **@path/to/other.md** 문법으로 분리된 파일을 참조 가능
    - 에이전트가 필요한 섹션만 selective하게 로드하도록 설계 가능

# Context Engineering

## ❖ 계층적 메모리 구조

- 서로 다른 지속성·접근 비용·용량을 가진 **저장소**를 **계층화**해서, 매 턴마다 필요한 정보만 context window에 **선별 주입**
- 파일 시스템**을 **외부 메모리**로 사용
- 에이전트가 직접 read/bash tool을 호출해 명시적으로 가져옴

영구적  
세션 시작 시  
고정 주입

context  
window

임시적

L1 — ~/.claude/CLAUDE.md  
사용자 전역 선호도, 스타일, 글로벌 gotchas

영구 / 세션 시작마다 자동 주입  
수동 편집으로 업데이트

L2 — ./CLAUDE.md (project root)  
아키텍처, 컨벤션, 워크플로우, known gotchas

프로젝트 수명 / 자동 주입  
git으로 팀 공유 가능

L3 — ./subdir/CLAUDE.md (module-level)  
모듈별 세부 규칙, 로컬 gotchas

해당 디렉토리 작업 시만 주입  
scoped injection

L4 — Session conversation history  
현재 세션 내 user ↔ assistant 턴

세션 종료 시 소멸  
sliding window eviction

L5 — Tool output accumulation  
bash 결과, file read, grep, test output  
agentic loop 반복마다 누적 → context 압박

세션 내 누적 / 명시적 파일 저장 시 영구  
filesystem = 외부 메모리 역할

L6 — Filesystem (explicit external memory)  
scratch files, todo.md, notes/, 중간 결과물

영구 / 명시적 Read/Write tool 호출  
context 밖에 존재

L7 — Working memory (scratchpad / thinking)  
extended thinking 블록, 중간 추론 과정

단일 추론 스텝 내 / 응답 후 소멸  
토론 사용 but 출력 안됨

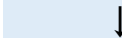
L1-L3 CLAUDE.md는 세션 시작 시 L4 상단에 고정 삽입됨 (budgeted, 고정 비용)  
L6 filesystem은 context 밖 — tool call로만 접근 (on-demand retrieval)

# Context Engineering

## ❖ 컨텍스트 압축과 요약 전략

- **절단(truncation)**
  - 오래된 turn을 잘라냄. 잘린 정보는 영구 소멸
- **요약(summarization)**
  - 대화가 **임계점**을 넘으면 LLM을 한 번 더 호출해 요약 생성
  - 요약은 자유형 텍스트보다 **구조화된 형식**이 훨씬 효율적
- **계층적 압축(progressive summarization)**
  - 요약을 **또 요약**하는 계층 구조
- **선택적 유지(selective retention)+RAG**
  - 모든 turn을 **vector DB**에 추가
  - 매 턴마다 현재 query와 관련된 과거 내용만 검색해 주입
- **[참고] Claude Code의 압축 전략**
  - Context window 사용량이 **일정비율 이상**  
⇒ 전체 대화 이력 요약 ⇒ context 대체

컨텍스트 ~95% 도달



현재 턴 일시 중지



전체 대화 히스토리 → 별도 모델 호출 → 구조화된 요약 생성



요약이 이전 턴 전체 대체



[Context compacted. Continuing from summary...] 표시



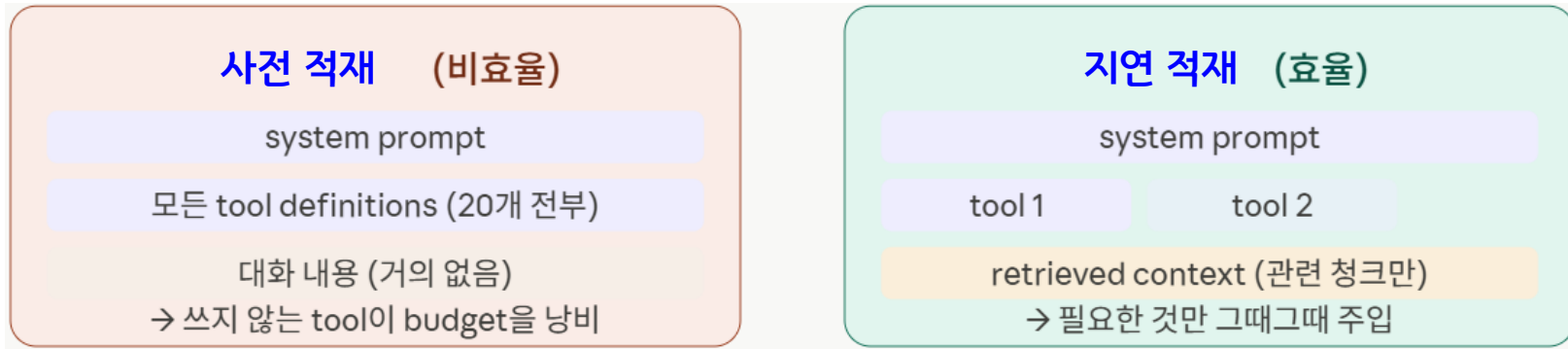
동일 태스크·파일 상태에서 세션 계속



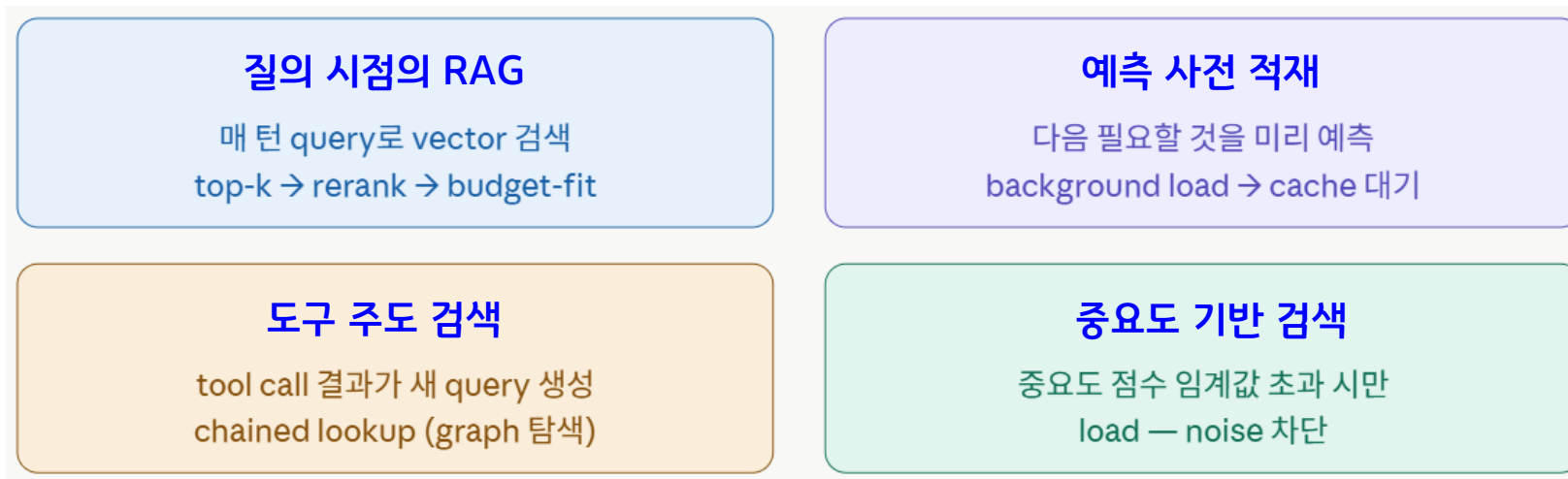
# Context Engineering

## ❖ 적재(loading)와 검색(retrieval)

- 사전 적재(eager loading)와 지연 적재(lazy loading)



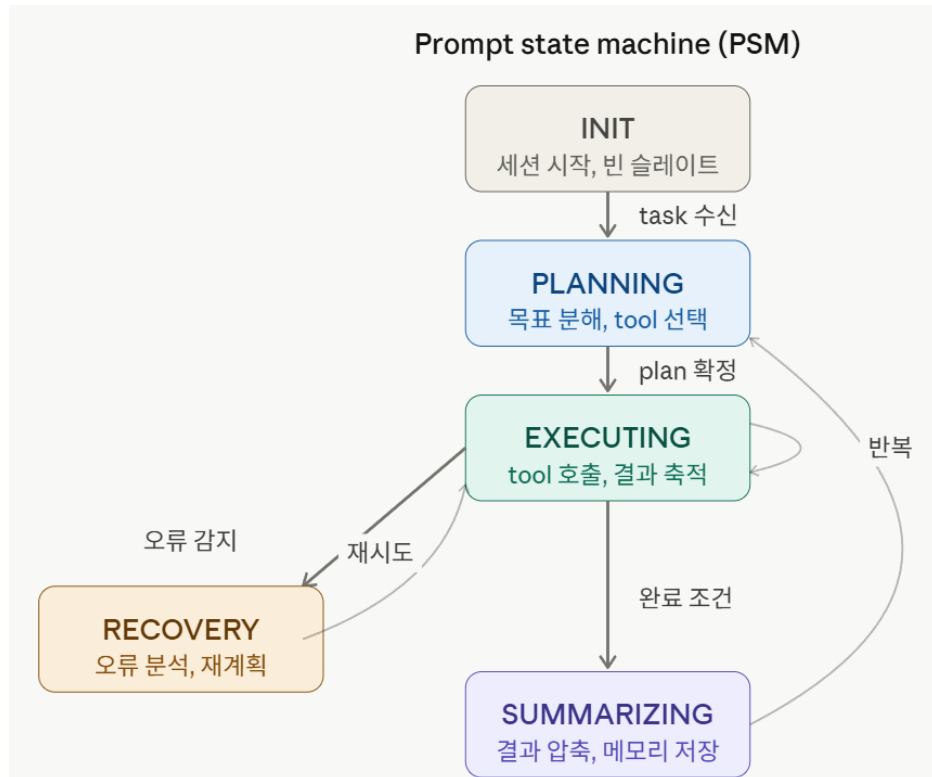
- 검색 패턴



# Context Engineering

## ❖ 프롬프트 상태 관리(prompt state management)

- Context window의 내용을 에이전트의 현재 상태(state)에 따라 다르게 구성하는 패턴
- 모든 정보를 항상 넣는 것이 아니라, 지금 무엇을 하고 있는지 상태에 따라 context를 동적으로 조립
- 각 상태 전환 시 context를 해당 상태에 맞게 재구성 — 이전 상태의 노이즈 제거

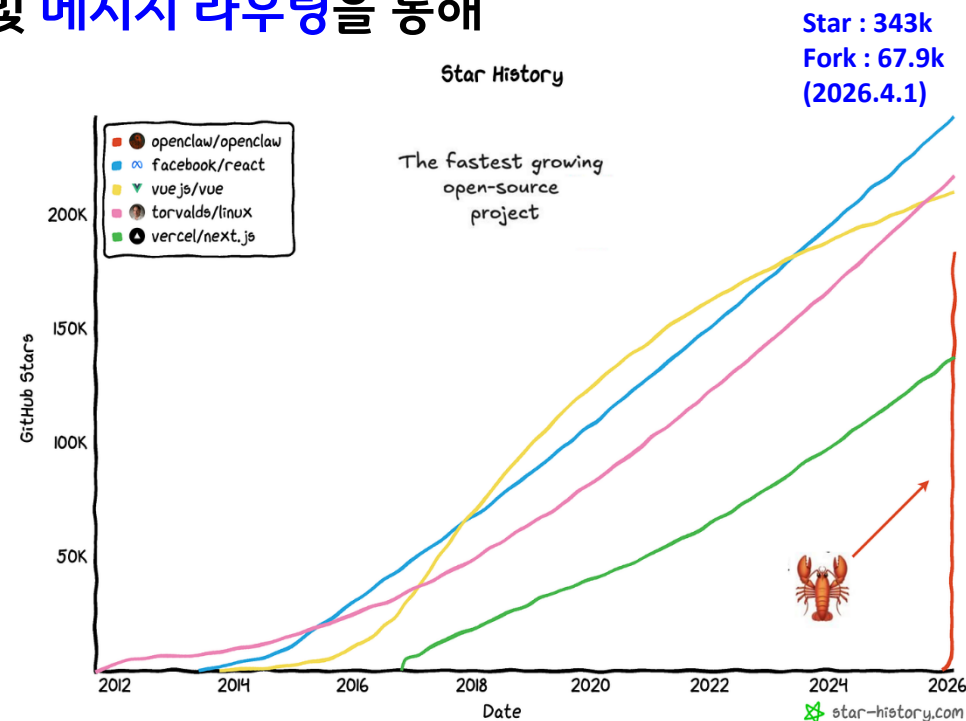


| State       | 포함                         | 제외 / 압축           |
|-------------|----------------------------|-------------------|
| INIT        | CLAUDE.md + system prompt  | 히스토리 없음           |
| PLANNING    | + task desc + tool schemas | tool 결과 아직 없음     |
| EXECUTING   | + plan + tool outputs      | 오래된 tool 결과 evict |
| RECOVERY    | + error trace + last state | 성공 path 압축        |
| SUMMARIZING | + 전체 결과 (최종 1회)            | 중간 과정 대부분 제거      |

## 4. OpenClaw

### ❖ OpenClaw

- Peter Steinberger 개발
- 로컬 환경에서 실행되는 강력한 런타임 능동적 비서 에이전트
- 모든 데이터와 실행환경을 사용자가 직접 소유하며 24시간 동작
- AI 에이전트를 위한 운영 체제
- 세션 관리, 메모리 시스템, 도구 샌드박싱(sandboxing) 및 메시지 라우팅을 통해 모델 주변에 구조화된 실행 환경 구축
- 실행되는 위치, 메시지 라우팅 방식, 사용할 수 있는 도구, 세션 격리 방식 등 직접 제어
- 대화 기록, 도구 실행, 세션 상태 및 모든 오케스트레이션 로직은 사용자 인프라에 유지

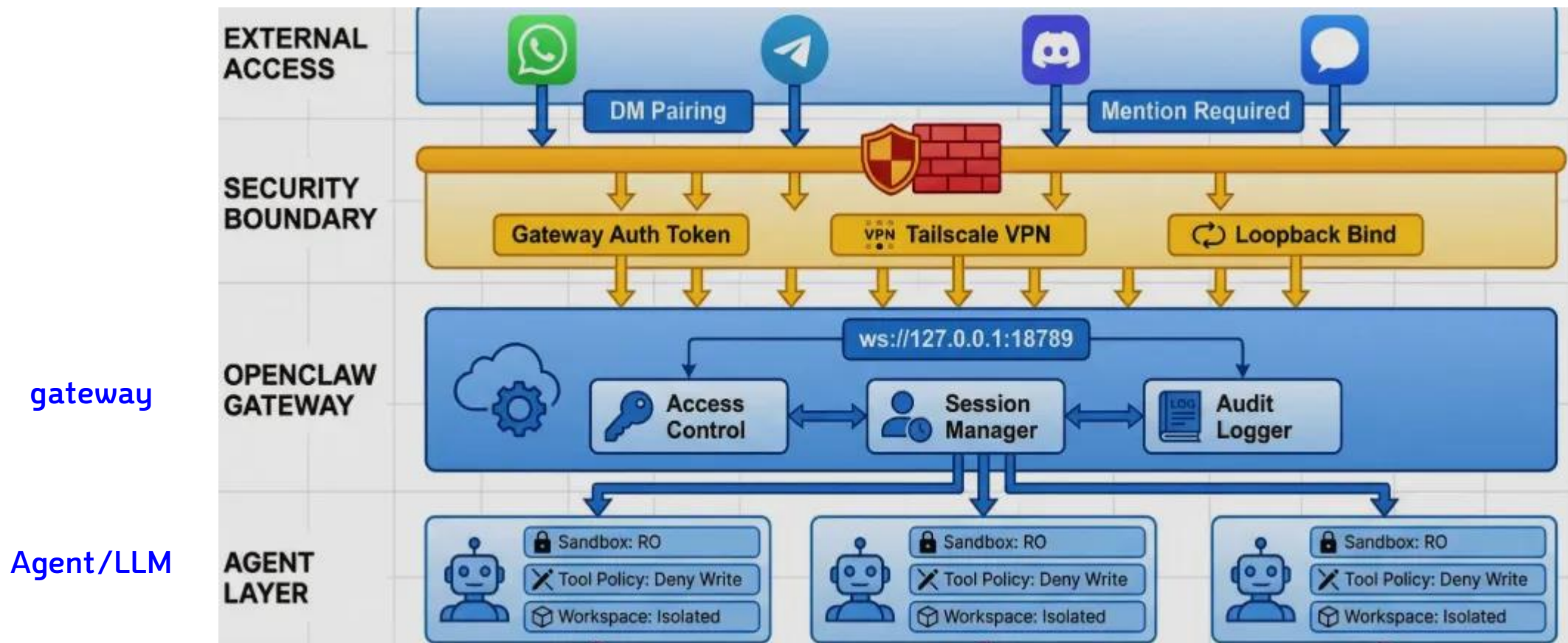


# 4.1 OpenClaw Architecture

## ❖ OpenClaw Architecture

- 단일 **게이트웨이**를 **중심**으로 하는 **hub-and-spoke** 아키텍처
- **Gateway**
  - 메시징 플랫폼 및 제어 인터페이스에 연결하여, 라우팅된 각 메시지를 **에이전트 런타임**으로 전달하는 **WebSocket 서버**
- **Agent runtime**
  - 종단간(end-to-end) **AI Loop 실행**
  - 세션 기록과 메모리에서 **컨텍스트 구성**
  - **모델 호출**
  - 시스템에서 사용 가능한 기능(브라우저 자동화, 파일 작업, 캔버스, 예약된 작업 등)에 대한 **도구 호출**
  - 업데이트된 **상태 저장**

# OpenClaw 구조



# OpenClaw

openclaw/

|                 |                                  |
|-----------------|----------------------------------|
| src/            | # 코어 소스 (~400K lines TypeScript) |
| index.ts        | # 진입점                            |
| gateway/        | # 게이트웨이 서버 (심장부)                 |
| boot.ts         | # 부트 시퀀스                         |
| server.impl.ts  | # 서버 구현                          |
| client.ts       | # WebSocket 클라이언트                |
| auth.ts         | # 인증 시스템                         |
| call.ts         | # RPC 라우팅                        |
| protocol/       | # WebSocket 프로토콜 정의              |
| server-methods/ | # RPC 핸들러들 (60+ 파일)              |
| chat.ts         | # 코어 메시지 핸들러                     |
| acp/            | # Agent Client Protocol          |
| translator.ts   | # 프로토콜 번역기                       |
| session.ts      | # 인메모리 세션 스토어                    |
| config/         | # 설정 시스템                         |
| config.ts       | # 설정 로딩                          |
| sessions/       | # 세션 스토어                         |
| store.ts        | # JSONL 세션 저장소                   |
| zod-schema.ts   | # Zod 스키마 검증                     |

<https://github.com/openclaw/openclaw>

# OpenClaw

|                         |                            |
|-------------------------|----------------------------|
| ├── plugins/            | # 플러그인 시스템                 |
| │   ├── discovery.ts    | # 플러그인 탐색                  |
| │   └── manifest.ts     | # 매니페스트 파싱                 |
| ├── plugin-sdk/         | # 플러그인 SDK (60+ 타입 export) |
| ├── cron/               | # 크론 스케줄러                  |
| │   ├── service.ts      | # CronService 클래스          |
| │   └── stagger.ts      | # Thundering Herd 방지       |
| ├── sessions/           | # 세션 유틸리티                  |
| ├── extensions/         | # 채널 플러그인 (45+ 패키지)        |
| │   ├── slack/          | # slack 봇                  |
| │   ├── telegram/       | # Telegram 봇               |
| │   ├── slack/          | # slack 봇                  |
| │   ├── whatsapp/       | # WhatsApp 클라이언트           |
| │   └── ...             | # 25+ 더 많은 채널              |
| ├── apps/               | # 네이티브 앱                   |
| │   ├── macos/          | # macOS 메뉴바 앱 (Swift)      |
| │   ├── ios/            | # iOS 컴패니언 앱 (Swift)       |
| │   └── android/        | # Android 컴패니언 앱 (Kotlin)  |
| ├── ui/                 | # 웹 UI                     |
| ├── test/               | # 테스트 인프라                  |
| ├── vitest.config.ts    | # 테스트 설정                   |
| ├── package.json        | # 프로젝트 메타데이터               |
| └── pnpm-workspace.yaml | # 모노레포 워크스페이스              |

## 4.2 Gateway

### ❖ Gateway

- 클라이언트(메신저 앱, 채널) 연결, 세션 및 메시지 라우팅을 관리하는 WebSocket 서버(기본 포트 18789)
  - 게이트웨이는 루프백(127.0.0.1)에 바인딩
  - CLI 도구, 웹 UI, 모바일 앱 모두 WebSocket 클라이언트로 연결
- 연결 상태 유지, cron 작업, 웹훅(webhook) 처리
- 시각적 출력을 위한 Canvas 호스팅
- 단일 Node.js 프로세스
- 메신저 플랫폼(Telegram, Slack, WhatsApp 등 20개 이상)와 연결
- 접근 제어
  - 루프백 바인딩을 제외한 모든 바인딩에 대해 토큰 또는 암호 인증을 통해 보안 강화
- 정규화된 메시지를 대응되는 세션으로 전달



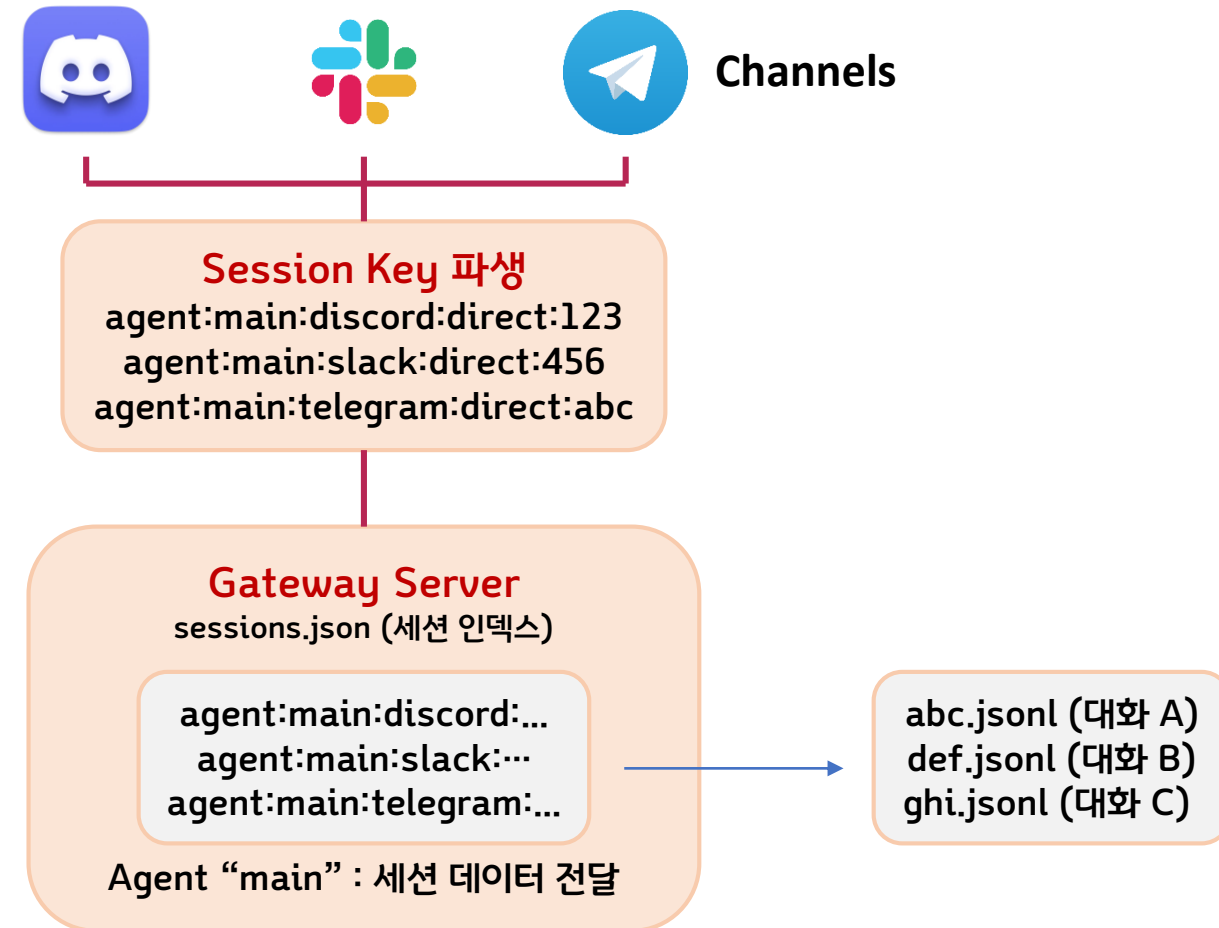
# Gateway — Channel & Session 관리

## ❖ 채널(channel)

- Telegram, Slack, WhatsApp 등  
게이트웨어와 연결된 **메시저 앱**
- 단순히 메시지가 오가는 통로

## ❖ 세션(session)

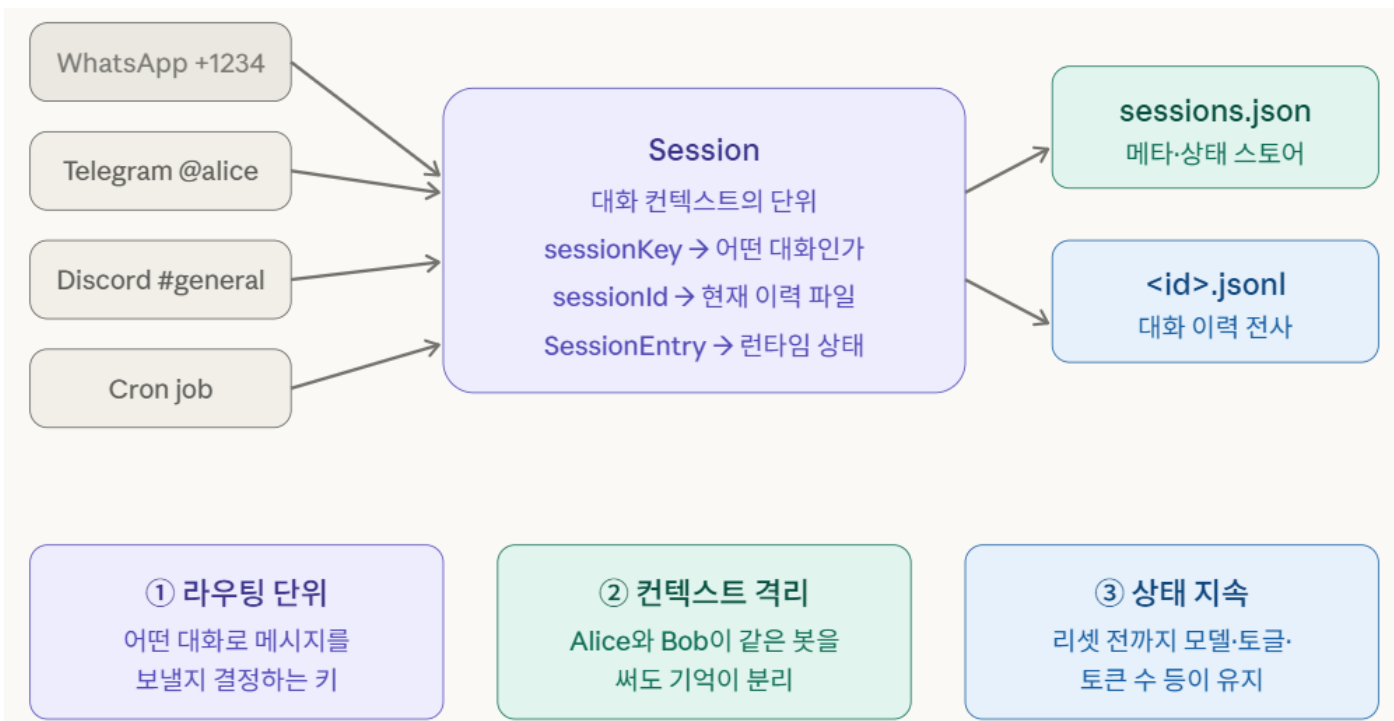
- 채널 내의 대화창 별로 세션 ID 생성
- 세션별 대화 이력 관리(파일 생성)
- 게이트웨이를 통해 에이전트에게 지시  
및 결과 확인
- **실제 대화**의 메모리와 상태를 **관리하는 주체**



# Gateway

## ❖ 세션

- "어떤 채널의 누구와의 대화"를 식별하는 **세션키(session key)** 사용
  - 라우팅과 격리 컨텍스트를 인코딩하는 구조화된 문자열 식별자
- **대화의 이력 파일과 런타임 상태**를 하나로 묶는 논리적 단위



# Gateway

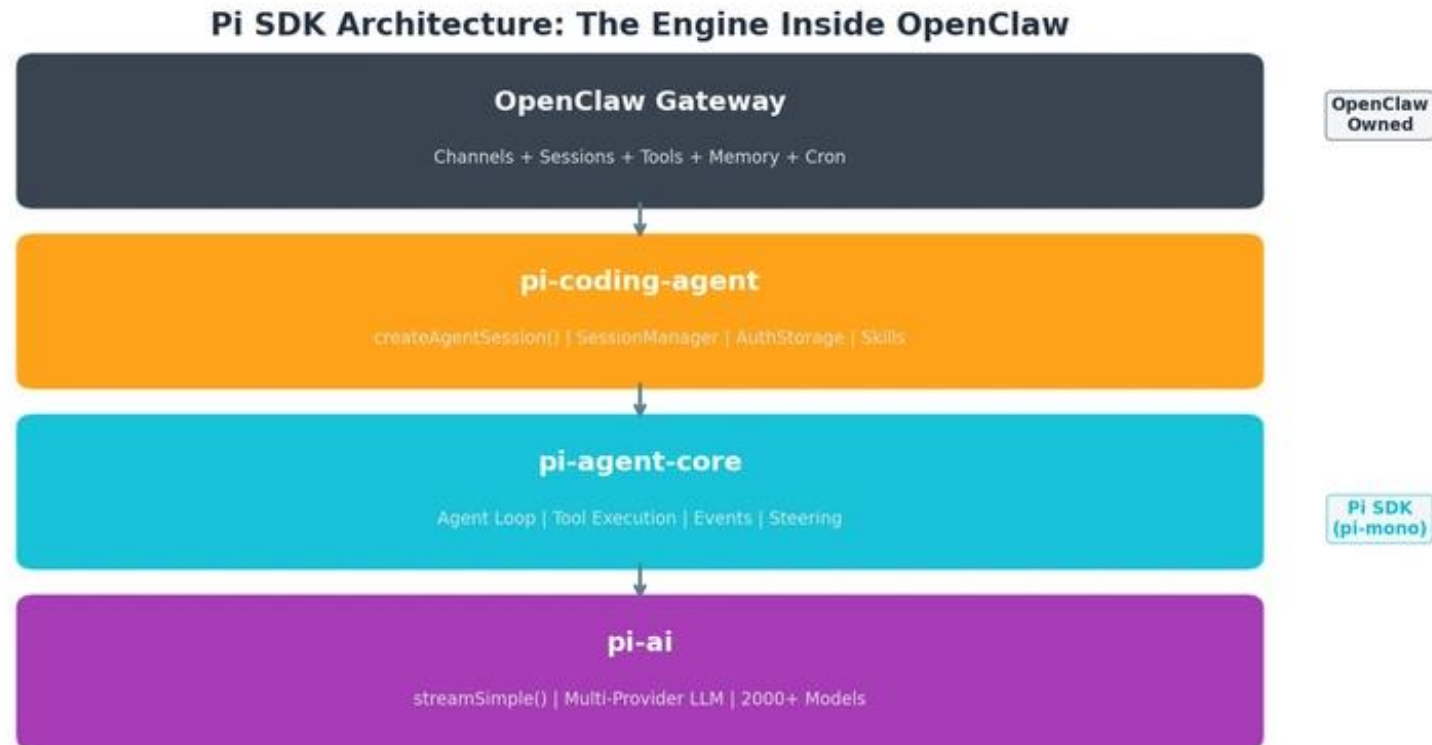
## ❖ 인증 메커니즘 (Authentication)

- 플랫폼별 인증방식 상이
- API 키 또는 OAuth(Claude Pro/ChatGPT 구독자의 경우)를 통해 가능
- 각 메시징 플랫폼별 채널 어댑터는 동일한 인터페이스 구현, 수신/발신 메시지 표준화
  - 인증 (Authentication)
  - 수신 메시지 구문 분석(Inbound message parsing)
  - 접근 제어(Access Control)
    - 허용목록(allowed list), DM(direct message), 기본 pairing, 그룹 정책 등
  - 발신 메시지 형식(Outbound message formatting)

## 4.3 Agent Runtime — Pi SDK 3계층

### ❖ 에이전트 실행환경(Agent Runtime) Pi SDK

- OpenClaw는 내부적으로 자체 에이전트 루프 미구현
- Mario Zechner가 만든 TypeScript 모노레포인 Pi SDK 내장



# Agent Runtime

## ❖ OpenClaw 에이전트 실행 환경

| 층  | 패키지             | 목적                                                                                                       |
|----|-----------------|----------------------------------------------------------------------------------------------------------|
| L1 | pi-ai           | LLM 핵심 추상화.<br>하나의 인터페이스로 2000개 이상의 LLM 모델 지원<br>streamSimple().completeSimple()                         |
| L2 | pi-agent-core   | 에이전트 루프(loop).<br>LLM으로 메시지 전송 → 도구 호출 → 결과 피드백 → 반복.<br>실행 중 중단 등 스티어링(steering) 및 후속 조치(후속 처리 대기열) 처리. |
| L3 | pi-coding-agent | 전체 런타임: createAgentSession(), SessionManager(트리 구조 분기 기능을 갖춘 JSONL 영구 저장), AuthStorage, 스킬, 확장 시스템 포함    |
| L4 | 오픈클로 게이트웨이      | 그 외 모든 것: 채널 어댑터, 세션 라우팅, 메모리, cron, heartbeat, 샌드박스, 다중 에이전트 라우팅, Canvas, 음성 및 WebSocket 제어 평면          |

# Agent Runtime

## ❖ Gateway 시작 — 이벤트 루프 초기화

node gateway.js 실행



[동기 실행 — Call Stack]

설정 파일 로드 (fs.readFileSync — 블로킹, 시작 시 1회만 허용)

WebSocket 서버 생성

→ uv\_tcp\_t Handle 생성 (:18789)

→ epoll에 소켓 등록

채널 어댑터 초기화

→ WhatsApp, Telegram 등 각 소켓 연결

→ uv\_tcp\_t Handle × N 생성

heartbeat 타이머 등록

→ uv\_timer\_t Handle 생성

cron 타이머 등록

→ uv\_timer\_t Handle × M 생성

applyGatewayLaneConcurrency()

→ 레인별 maxConcurrent 설정

Call Stack 비어짐



[이벤트 루프 진입]

poll 단계 → epoll\_wait 블로킹 대기

CPU 0% — 이벤트 올 때까지 대기

OUTER LOOP: while() / run.ts:815 — while(true) 재시도 루프  
(auth 실패, context overflow, 모델 failover 처리)

ATTEMPT: run/attempt.ts — 1회 실행 시도  
(시스템 프롬프트 조립, 세션 로드, 도구 등록)  
StreamFn 래퍼 추가, createAgentSession({...}) ← L3 진입

L3 : pi-coding-agent

- SessionManager (JSONL 영속화)
- session.prompt() 인터페이스

L2 : INNER LOOP: pi-agent-core

LLM 호출 → tool call → 결과 → LLM 재호출  
(end\_turn 또는 abort까지 반복)

L1 : pi-ai

Anthropic/OpenAI/Google

# Agent Runtime

## ❖ 메시지 실행 파이프라인

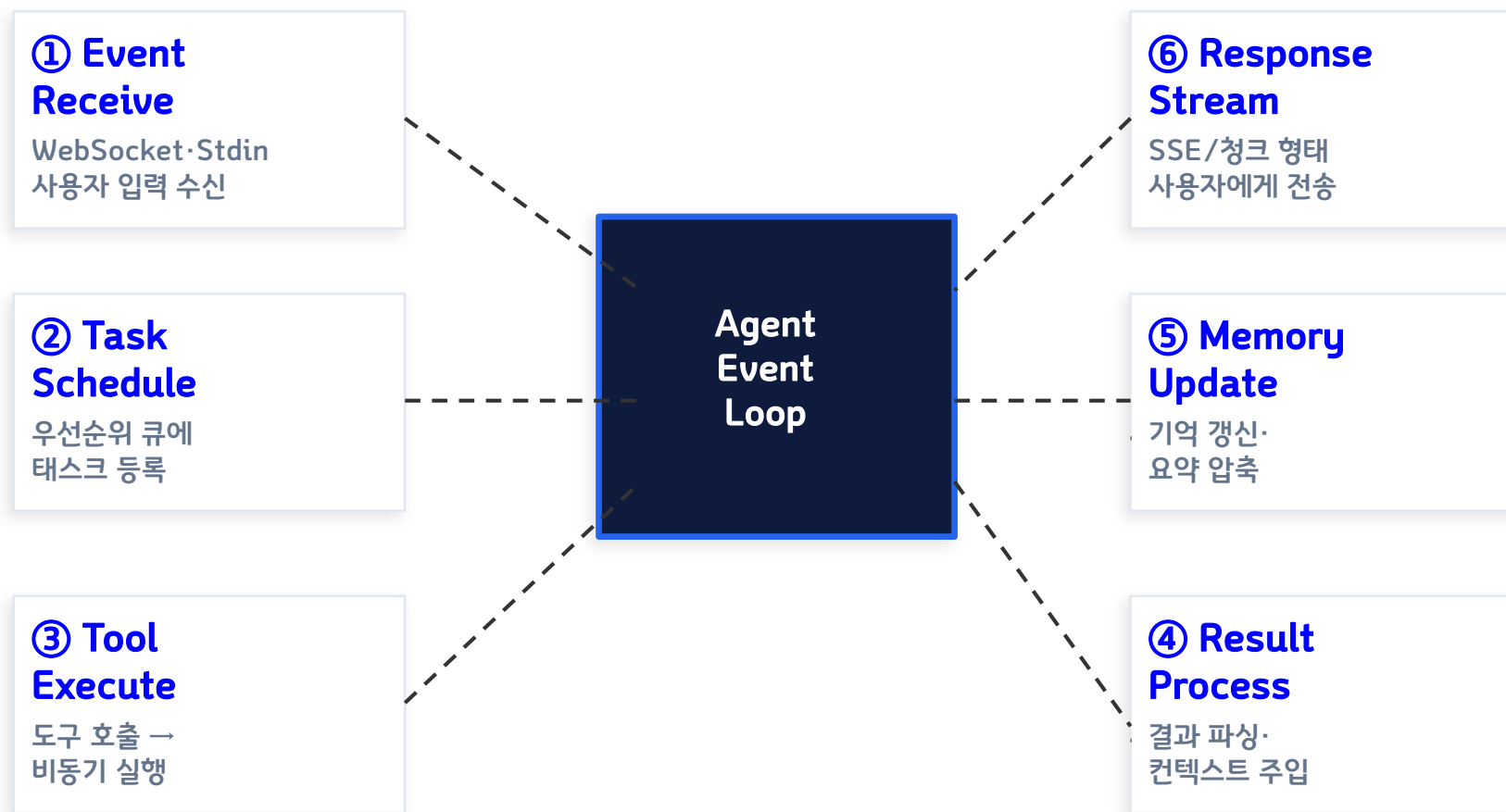
- 채널에서 전송되는 모든 메시지는 엄격하게 정의된 6단계 파이프라인으로 처리



## 4.4 Agent Loop

### ❖ 에이전트 이벤트 루프(event loop)

- 이벤트 기반(event-driven) 방식 처리
- 단일 프로세스로 처리 (외부 도구, LLM, IO는 별도 프로세스)





# Agent Loop

## ❖ 실행 환경

- **Node.js** 이벤트 루프 기반 큐(queue)
- **libuv event loop** 사용
- JavaScript 비동기 처리 사용
  - **Promise** 객체: **async/await**

async function 호출



await 지점 도달 → Promise 큐에 콜백 등록  
→ 함수 실행 일시 중단  
→ 콜 스택 반환 (다른 코드 실행 가능)

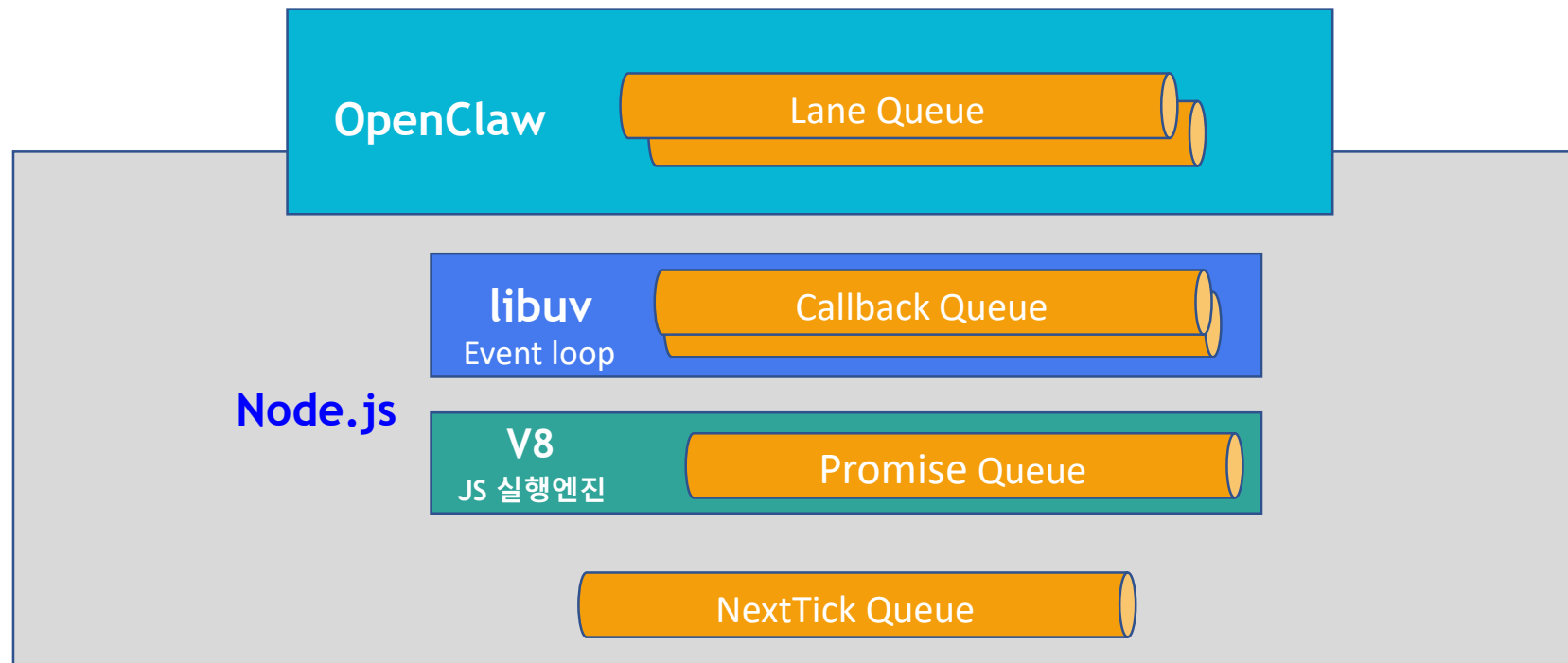


Promise resolve → Promise 큐에서 꺼내어 실행 재개

# Agent Loop

## ❖ 실행 환경 – cont.

- Node.js 와 OpenClaw 관계



# Agent Loop

## ❖ Queue System

### ▪ Lane Queue

- TypeScript로 구현된 **세마포어**
- 에이전트 실행의 순서와 동시성 제어

Lane Queue (TypeScript - command-queue.ts)

|                |                    |                |                            |
|----------------|--------------------|----------------|----------------------------|
| Main<br>max: 4 | Subagent<br>max: 8 | Cron<br>max: 1 | session:<key> *N<br>max: 1 |
|----------------|--------------------|----------------|----------------------------|

Lane Queue 맵 (command-queue.ts):

```
Map<string, LaneState>
```

|                                                |                       |             |   |    |
|------------------------------------------------|-----------------------|-------------|---|----|
| "main"                                         | Global Lane - Gateway | 시작          | 시 | 생성 |
| "subagent"                                     | Global Lane - Gateway | 시작          | 시 | 생성 |
| "cron"                                         | Global Lane - Gateway | 시작          | 시 | 생성 |
| "nested"                                       | Global Lane - Gateway | 시작          | 시 | 생성 |
| "session:agent:main:whatsapp:dm:+821012345678" |                       | ← 첫 메시지     | 시 | 생성 |
| "session:agent:main:telegram:dm:12345"         |                       | ← 첫 메시지     | 시 | 생성 |
| "session:agent:main:discord:group:987654"      |                       | ← 첫 메시지     | 시 | 생성 |
| "session:agent:id:subagent:uuid-A"             |                       | ← spawn     | 시 | 생성 |
| "session:agent:id:subagent:uuid-B"             |                       | ← spawn     | 시 | 생성 |
| ...                                            |                       | ← 세션 수만큼 증가 |   |    |

# Agent Loop

## ❖ Queue System – cont.

### ▪ Callback Queue

- 외부 세계(OS, 스레드 풀)가 보내온 완료 이벤트를 받는 대기열(queue)

### ▪ Microtask Queue

- nextTick queue
  - `process.nextTick()`으로 예약된 콜백
- Promise queue
  - 받은 이벤트의 결과를 코드 안에서 전달하는 통로
  - `Promise.then` / `async-await`의 후속 처리

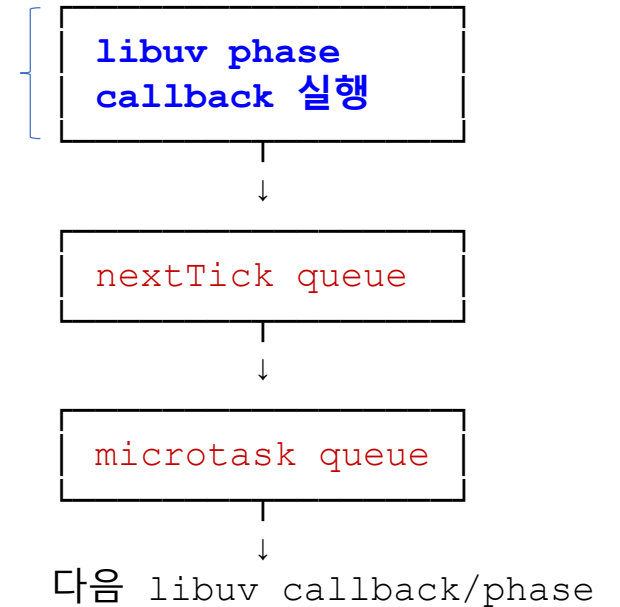
### ▪ 실행 우선순위

- nextTick queue > Promise queue > Callback queue

# Agent Loop

## ❖ 이벤트 루프(event loop) 라이브러리 libuv

- timers 큐
  - setTimeout / setInterval 만료 콜백  
→ heartbeat, cron, subagent timeout
- pending 큐
  - 이전 tick의 poll 단계에서 처리하지 못하고 미뤄진 I/O 콜백
  - 주로 시스템 레벨 오류 콜백. "처리 못하고 넘어온 것"
- poll 큐
  - 새로 완료된 I/O 이벤트 처리. "이번에 새로 온 것"
  - 소켓 데이터 도착 콜백 (WebSocket, LLM API, 채널)
  - 스레드 풀 완료 콜백 (JSONL 파일 읽기/쓰기) → 가장 많은 OpenClaw 이벤트가 여기서 처리됨
- check 큐
  - setImmediate 콜백
- close 큐
  - close 이벤트 콜백, socket.destroy() 등



# Agent Loop

## ❖ 이벤트 루프(event loop) 라이브러리 libuv – cont.

- libuv의 event loop에서 nextTick과 Promise Queue의 event 처리
  - AsyncWrap::MakeCallback
    - Node의 거의 모든 native 비동기 객체가 AsyncWrap를 상속
    - 이 메서드가 InternalMakeCallback(env(), object(), cb, argc, argv, {async\_id, trigger\_async\_id})를 호출
      - » nextTickQueue를 전부 drain
      - » v8::Isolate::PerformMicrotaskCheckpoint() 호출 → V8이 microtask queue를 전부 drain

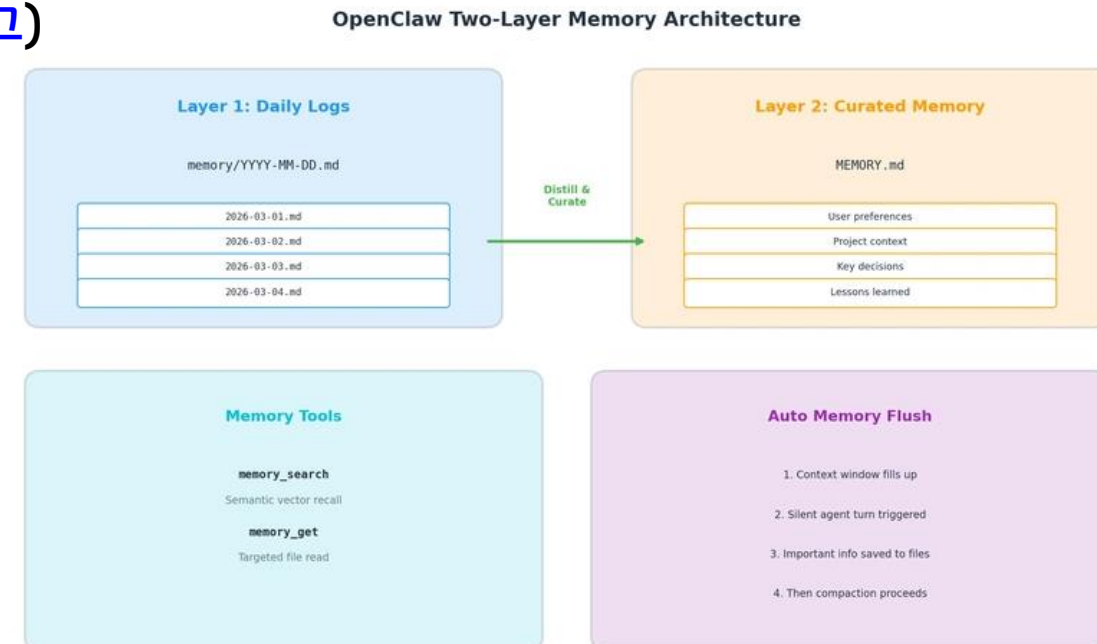
```
uv_run()    (event loop)
└─ poll/check/timers phase에서 등록된 c 콜백 실행
    예: AfterConnect(uv_connect_t* req, int status)
        └─ wrap->MakeCallback(oncomplete_fn, argc, argv)    [AsyncWrap::MakeCallback]
            └─ InternalMakeCallback(env, wrap->object(), oncomplete_fn, ...)
                └─ InternalCallbackScope scope(...);
                    oncomplete_fn->Call(...);    // ← JS 진입
                    scope.Close();                // ← (outermost면) nextTick + microtask drain
```

# 4.5 Memory Systems

## ❖ 메모리

- 작업 공간(Workspace)에 저장된 **평문 Markdown** 파일
- 파일 자체가 정보의 원천
- 2-Layer 메모리 구조
  - Layer 1: **MEMORY.md** (**장기 기억**)
    - 결정사항, 선호도, 지속적으로 참조해야 할 사실
    - 개인 세션에서만 로드 (그룹 채팅에서는 로드 안 됨)
  - Layer 2: **memory/YYYY-MM-DD.md** (**일일 로그**)
    - 날짜별 로그, 추가 전용(append-only)
    - 세션 시작 시 오늘 + 어제 데이터가 자동 로드
    - 일종의 "단기-중기 메모리" 역할

```
memory/  
├── 2026-03-01.md  
├── 2026-03-02.md  
├── 2026-03-03.md  
└── 2026-03-04.md
```



# Memory Systems

## ❖ SQLite + Markdown 하이브리드 지원

- 저장은 Markdown, 검색은 SQLite
  - 에이전트는 워크스페이스에 Markdown 파일 그대로 유지
  - 검색용 인덱스만 **SQLite**(+sqlite-vec 확장)에 따로 구축
- 자동 인덱싱
  - Gateway는 파일 시스템을 감시하다가 .md 변경을 감지하면 자동으로 **SQLite 벡터 저장소**에 재인덱싱
- **벡터 검색 성능** (sqlite-vec)
  - Available: 임베딩이 SQLite virtual table(vec0)에 저장 - 빠름
  - Unavailable: 작동은 하지만 대규모 메모리에서 느려짐
- OpenClaw는 RAM에 정보를 유지 하지 않음
  - 파일에 기록해야 세션 종료 후에도 남음

```
~/.openclaw/workspace/      # 기본 워크스페이스
├── AGENTS.md                 # 에이전트 설정
├── BOOTSTRAP.md              # 초기화 프롬프트
├── HEARTBEAT.md              # 주기적 체크 작업
├── IDENTITY.md               # 에이전트 정체성
├── MEMORY.md                 # 장기 기억 (핵심)
├── SOUL.md                   # 성격 및 태도
├── TOOLS.md                  # 도구 설정
├── USER.md                   # 사용자 정보
├── canvas/                   # 작업 디렉토리
├── memory/                   # 일일 로그 디렉토리
│   ├── 2026-03-26.md
│   ├── 2026-03-27.md
│   └── 2026-03-28.md
```



# Memory Systems

## ❖ 프롬프트(prompt) 구성

- 세션 메모리 + 워크스페이스 메모리 조합 포함

```
runEmbeddedPiAgent()  
  enqueueSession() =>      # (Session Actor Queue로 직렬화)  
    enqueueGlobal(async () => {  
      runEmbeddedAttempt() → run/attempt.ts
```

[A] 워크스페이스 메모리 로드 ( MEMORY.md, SOUL.md 등), 도구 등록 등(bash)

[B] 시스템 프롬프트 조립

[C] 세션 이력(JSONL) 로드

[D] 메모리 검색 결과 (옵션)

```
  resolveMemorySearchConfig()  
    sources: ["memory"] 또는 ["memory", "sessions"]
```

[E] 최종 페이로드 조립 → LLM 전송

```
  createAgentSession({  
    systemPrompt: [B], ← 워크스페이스 메모리 포함  
    messages: [C], ← 세션 히스토리 (JSONL)  
    prompt: 유저입력,  
  })
```

[d] streamFn 래퍼 체인 장착  
(각 래퍼는 LLM 응답 스트림 가로채서 정규화함)

## 4.6 Subagent

### ❖ Subagent(하위 에이전트)

- 부모 에이전트가 **격리된 자식 에이전트**를 **spawn**하여 위임 작업을 수행시키는 메커니즘
- 독립된 세션 컨텍스트, 제한된 도구 정책(restricted tool policies), 깊이 제한(depth limits)
- 부모의 큰 컨텍스트는 복사되지 않음 (자식은 가벼운 컨텍스트로 시작)
- 무한 재귀 방지를 위한 depth limit (기본 2)
- **sessions\_spawn**
  - 새 **subagent 세션**을 생성하고 task를 위임
  - agentId, model, runtime 지정 가능
- **sessions\_yield**
  - 현재 모델 턴(turn)을 **종료**하고 **런타임 이벤트**(주로 subagent 완료)를 기다림  
→ **polling 루프** 대신 사용해야 함 (sessions\_list 반복 호출 등은 금지)
- **sessions\_send / sessions\_history**
  - 에이전트 간 **메시지 송신** 및 **이력 조회** (부모-자식 / 형제 통신)

# Subagent

## ❖ Subagent 호출 시 프롬프트 구성

### ① Task Description (사용자가 명시한 작업)

- sessions\_spawn 호출 시 task 인자로 전달
- 예. "Analyze PR #42 for bugs", "Research three competitors"

### ② Inherited Workspace Context

- 해당 subagent의 워크스페이스에서 AGENTS.md, SOUL.md 등 자동 로드  
→ 부모와 다른 페르소나 가능 (예: 부모=Orchestrator, 자식=Researcher)

### ③ Restricted Tool List

- 부모의 도구 목록에서 정책에 따라 일부 제거된 상태
- /tools 명령으로 effective tool list 확인 가능

### ④ Active Subagents 블록 (부모 측)

- 부모의 매 턴에 자동 주입되는 런타임 생성 블록.
- 현재 활성 자식 세션, runId, 상태 정보를 부모가 추적할 수 있도록 함

### ▪ 기본 모델/thinking 설정은 caller(부모)로부터 상속

- agents.defaults.subagents 설정으로 오버라이드(override) 가능

# Subagent

## ❖ Subagent의 호출 및 결과 반환

### ▪ Spawn → Yield → Resume 패턴

#### ① 부모: sessions\_spawn 호출

- 부모는 즉시 {runId, sessionId}를 받음.
- 자식은 background 시작

```
sessions_spawn({  
  task: "Analyze the latest 3 competitor pricing pages",  
  agentId: "researcher",  
  runTimeoutSeconds: 600  
})
```

#### ② 부모: sessions\_yield로 대기 sessions\_yield()

- 부모 모델 턴 종료. Gateway가 자식 완료 이벤트를 기다림
- 결과 도착 시 다음 메시지로 자동 주입

#### ③ 자식: 자체 Agent Loop 실행

- 독립된 INNER LOOP로 task 수행. 결과를 final message로 emit

#### ④ 자식 ⇨ 부모로 결과 전달

- subagent 완료 이벤트가 부모 세션에 메시지로 주입됨
- 부모는 yield에서 깨어나 다음 추론 진행

## 4.7 Workspace

### ❖ 워크스페이스(Workspace)

- 각 에이전트의 **성격·말투·행동 방식**을 정의하는 디렉토리(보통 ~/.openclaw/workspace/).
- 같은 Claude 모델을 쓰더라도 workspace 내 .md 파일들에 따라 완전히 다른 캐릭터의 에이전트
- 매 대화마다 자동 로딩되는 파일
  - AGENTS.md, SOUL.md, USER.md, TOOLS.md, IDENTITY.md 등이 자동 로딩.
- MEMORY.md, BOOTSTRAP.md, HEARTBEAT.md 등은 별도 로직으로 처리
- 다중 에이전트(multi-agent)의 경우
  - 한 OpenClaw 내의 각 에이전트는 별도의 workspace 보유
  - ~/.openclaw/agents/<agentId>/workspace/
- 에이전트 간 대화하려면 상대방의 ID(Slack User ID, Telegram ID 등) 필요

```
sessions_send({  
    targetAgent: "ai_CTO",  
    message: "신규 기능 x에 대한 기술 검토 부탁"  
})
```

# Workspace

## ❖ Workspace 내의 파일

📁 정체성 · 설정 파일

- 📄 **SOUL.md** 나는 누구인가 성격
- 👤 **USER.md** 집사님은 누구인가 팀원
- 📅 **AGENTS.md** 일하는 방법 규칙
- 🔧 **TOOLS.md** 도구 사용 설명서 API
- 💖 **HEARTBEAT.md** 일과표 루틴
- 📄 **IDENTITY.md** 프로필 카드 봄베이
- 📝 **MEMORY.md** 장기 기억 핵심

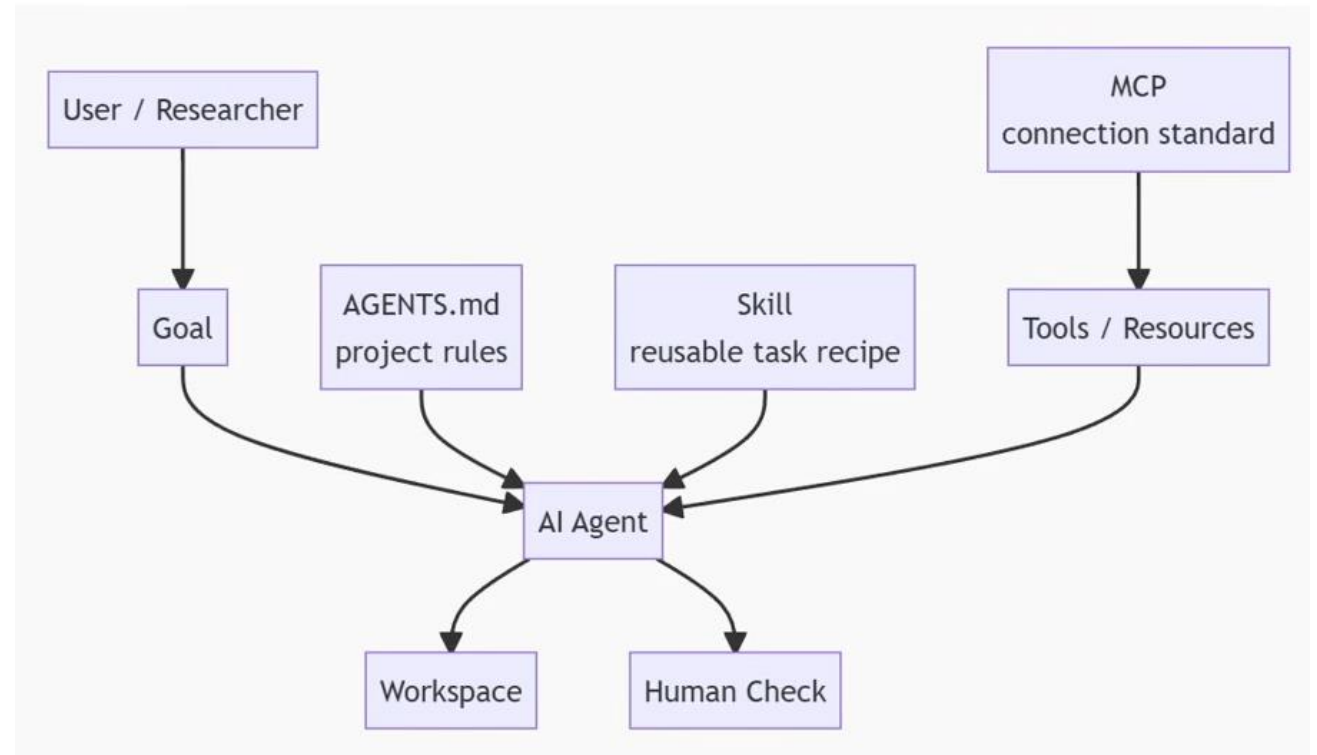
📁 업무 폴더

- 📁 **memory/** 일일 작업 일지 Day1~5
  - 📄 2026-04-02-1215.md
  - 📄 2026-04-02-1425.md
  - 📄 2026-04-22-1043.md
- 📁 **context/** 운영 가이드 문서
  - policies.md, playbooks.md, about-ai-study.md
- 📁 **operations/** 진행 중인 기수 21기
  - 운영정보.md, 진행현황.md, artifacts/
- 📁 **scripts/** 자동화 스크립트
- 📁 **sessions/** 대화 기록 268개
- 📁 **archives/** 종료된 기수 보관

## 4.8 Multi-Agent

### ❖ Multi-Agent

- 여러 에이전트 생성 및 사용
- 에이전트
  - .md 파일에 정의된 에이전트 프로파일
  - 별도 workspace 보유



# Multi-Agent

## ❖ Channel-Agent Binding

### ▪ 결정론적 라우팅 (Deterministic Routing)

- OpenClaw에서 "어떤 메시지가 어느 에이전트에게 가야 하는가"는 모델 미개입
- Host 설정 파일의 bindings 규칙으로 결정론적으로 라우팅

### ▪ Binding 구조

- binding은 (channel, accountId, peer/guild) 튜플 → agentId 매핑

```
{ "bindings": [  
  { "agentId": "work",  
    "match": { "channel": "slack", "accountId": "company" } },  
  { "agentId": "personal",  
    "match": { "channel": "whatsapp",  
              "peer": { "kind": "direct", "id": "+15551230001" } } }  
]
```

### ▪ Matching 규칙 (Tiered)

- 라우팅의 확인 순서
  - ① peer/guild 매칭 (가장 구체적) ② accountId 매칭 ③ channel 매칭 (가장 일반적)
- 가장 구체적인 것에 binding (Most-specific wins)



# Multi-Agent

## ❖ Channel-Agent Routing

- 멀티 에이전트, 단일 Gateway
  - 하나의 Gateway 프로세스가 **여러 에이전트**(work / personal / ops 등) 운영
  - 각 에이전트는 자체 workspace, agentDir, sessions 보유
- 운영 예시
  - 하나의 Telegram 봇으로 여러 페르소나 분기
    - Telegram DM → **main 에이전트** (기본 동작)
    - Telegram Group
      - Codex topic → **codex 에이전트** → ACP → Codex CLI
      - Claude topic → **claude 에이전트** → ACP → Claude Code

```
openclaw agents list --bindings          # 모든 에이전트와 binding 조회
openclaw agents add <id> --workspace <path> # 새 에이전트 생성
openclaw agents bind --agent <id> --bind <channel>:<criteria> # binding 추가
```

## 4.9 Extension Mechanism

### ❖ MCP

- 여러 MCP 서버를 동시에 연결해 도구 사용
- 각 MCP 서버는 특정 서비스(Gmail, GitHub, Notion 등)를 노출하는 경량 프로세스
- 사용 모드
  - Outbound MCP
    - OpenClaw가 클라이언트
    - openclaw.json의 mcpServers 블록에 외부 서버 등록 → OpenClaw가 그 도구를 사용
  - Inbound MCP
    - OpenClaw 자체가 MCP 서버 역할 (openclaw mcp serve)
    - Claude Code/Codex/Cursor 등 외부 MCP 클라이언트가 OpenClaw의 채널 대화에 접근할 수 있게 함

```
openclaw mcp list / show / set / unset
```

# Extension Mechanism

## ❖ Hook

- 디렉토리 자동 발견 방식
  - **HOOK.md**(메타데이터) + **handler.ts**(핸들러 로직).
- **Internal Hook**
  - Gateway 내부 이벤트
  - /new, /reset, /stop, 세션 컴팩션, 에이전트 부트스트랩, 게이트웨이 시작 등
- **Webhook**
  - 외부 HTTP 트리거
  - 다른 시스템이 OpenClaw에서 작업을 트리거할 수 있게 함 (Gmail Pub/Sub 등)
- **주요 이벤트**
  - session:start / message:received / message:sent / tool:before / tool:after
  - session:compact:before / session:compact:after / agent:bootstrap
  - gateway:startup / gateway:shutdown / gateway:pre-restart
  - command:new / command:reset / command:stop

## 4.10 Proactive Mechanisms

### ❖ Cron

- **게이트웨이 내장 스케줄러. 특정 시각/주기에 고정된 작업 실행**
  - 예. "Use the cron schedule tasks tool to send me a daily summary every morning at 9 AM" → 에이전트가 내장 cron 도구를 호출해 작업 등록
  - 메인 세션 또는 격리된 세션에서 실행 가능
- 결과를 채팅 채널로 announce 가능
- 일회성(one-shot) reminder(--at) 지원
- 별도 cron 레인(lane)에서 직렬화되어 메인 트래픽 방해 안 함

```
openclaw cron list      # 모든 cron job 보기
openclaw cron edit <name>      # 수정
openclaw cron remove <name>    # 제거
```

- cron 실행 시점은 서버 타임존 기준 (로컬 시계 아님). 항상 시간대 확인 필요

# Proactive Mechanisms

## ❖ Heartbeat

- 메인 세션에서 주기적으로(기본 30분) 도는 에이전트 "turn".
- HEARTBEAT.md 체크리스트를 읽고 "뭔가 처리할 게 있는가?"를 판단해 행동

`openclaw system heartbeat enable / disable / last`

- HEARTBEAT.md 동작 방식
  - 파일에 체크리스트("디스크 85% 초과 시 알림" 등) 작성
  - 30분마다 자동 실행, 이상 없으면 "HEARTBEAT\_OK" 응답하고 조용히 종료
  - 파일이 비어있으면 API 호출 자체를 스킵 (비용 절감)
- Quiet Hours / Active Hours
  - 특정 시간대(예: 새벽)에는 자동 억제 가능
  - Heartbeat turn은 task record를 만들지 않고 일일/idle 세션 갱신에도 영향 없음

## 4.11 도구 사용

### ❖ 도구 사용(tool\_use)

- LLM이 **도구 호출이 필요하다고 판단**했을 때 assistant 메시지의 content 배열 안에 포함되는 **JSON 형태의 구조화된 도구 호출 명세 블록**
  - {"type": "text"} LLM의 자연어 출력 → 사람이 읽음
  - {"type": "tool\_use"} LLM의 도구 호출 명세 → 코드가 파싱하여 실행
- 도구 실행 후, 결과를 다음 메시지에 담아 전송

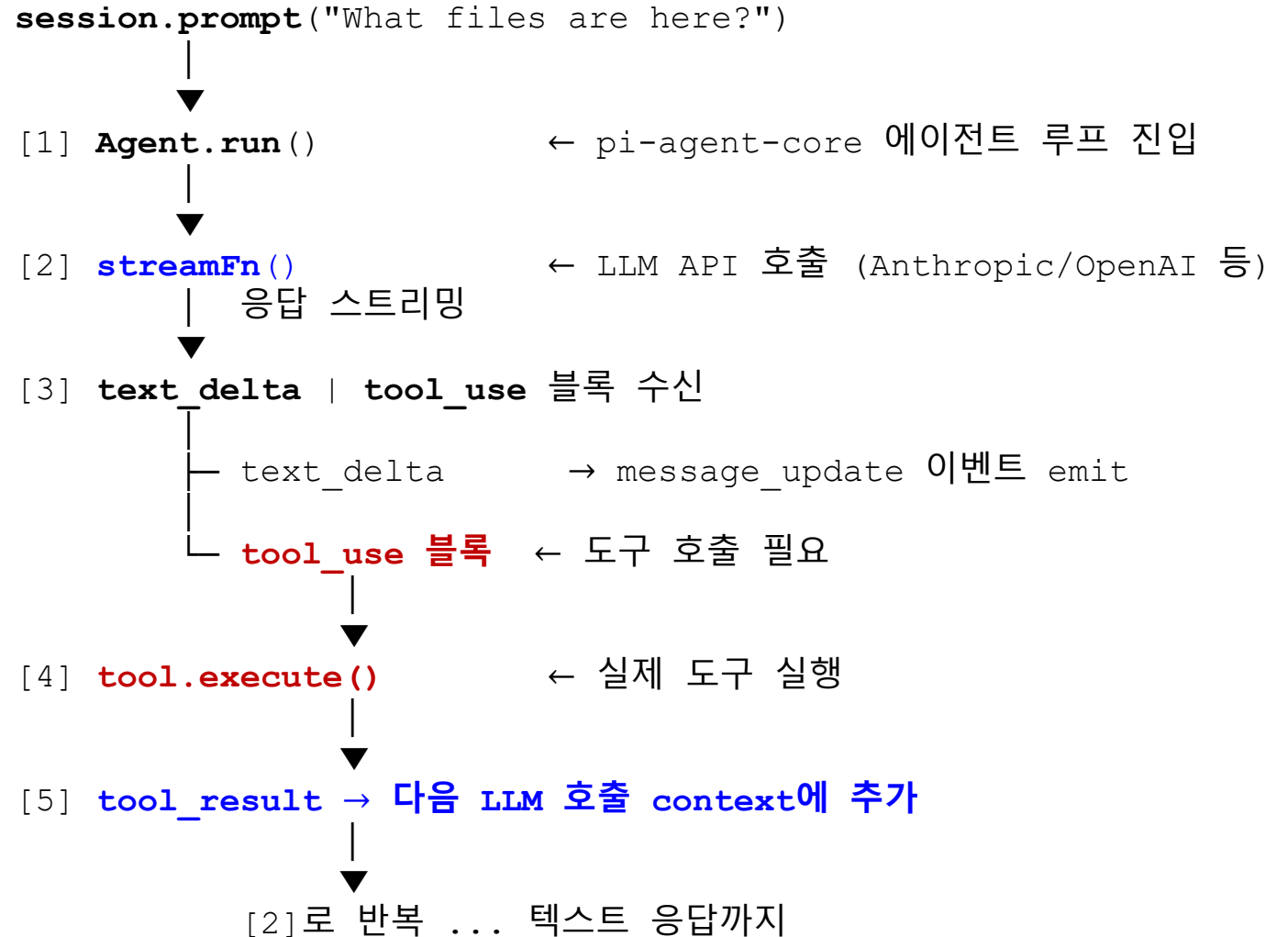
```
{
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "날씨를 확인해 볼게요."
    },
    {
      "type": "tool_use",
      "id": "toolu_01AbCdEf",
      "name": "get_weather",
      "input": { "location": "Seoul" }
    }
  ]
}
```

이건명, 충북대

```
messages = [
  {"role": "user", "content": "서울 날씨 알려줘"},
  # LLM의 tool_use 응답 (그대로 포함)
  {"role": "assistant", "content": [
    {"type": "tool_use", "id": "toolu_01AbCdEf",
      "name": "get_weather", "input": {"location": "Seoul"}}
  ]},
  # 실행 결과를 tool_result로 반환
  {"role": "user", "content": [
    {
      "type": "tool_result",
      "tool_use_id": "toolu_01AbCdEf", # ← id 일치시켜야 함
      "content": "현재 서울은 22°C, 맑음"
    }
  ]}
]
```

# 도구 사용

## ❖ tool\_use 시퀀스



## 4.12 계획 수립

### ❖ 계획 수립(planning)

- 별도의 **planning** 모듈 부재

- **Prompt 조립** → **모델 추론** → **Tool Call** 루프의 연쇄로 **암묵적으로 수행**

User Prompt (인바운드 메시지)

▼ [1] Directive 파싱

parseReplyDirectives() + detectCommand()

| /think, /model, /verbose, /new 등 처리

▼ [2] System Prompt 조립 ← "플랜 환경" 구성

buildEmbeddedSystemPrompt()

└─ buildAgentSystemPrompt()

| 도구 목록, 워크스페이스, 스킬, 부트스트랩 파일 주입

▼ [3] 모델 추론 ← "플랜 생성" 발생 지점

LLM (첫 번째 응답)

└─ <think>...</think> 내부 추론 (reasoning 활성화 시)

→ 텍스트 계획 or 즉시 tool\_use 블록

|

▼ [4] Tool Call 루프 ← "플랜 실행"

subscribeEmbeddedPiSession()

└─ tool\_use → 도구 실행 → tool\_result → 다음 모델 호출

| 반복

▼ [5] 최종 응답 조립

buildReplyPayloads() → 채널로 전달



# 계획 수립

## ❖ 모델 추론 실행 코드 전체 경로

```
runEmbeddedPiAgent()    ← run.ts    진입점
├─ runEmbeddedAttempt() ← run/attempt.ts 실제 모델 호출
│
│   └─ [1] SystemPrompt 생성
│       └─ [2] Pi Session 생성 (createAgentSession)
│           └─ [3] SystemPrompt 주입 (applySystemPromptOverrideToSession)
│               └─ [4] 이벤트 구독 (subscribeEmbeddedPiSession)
│                   └─ [5] 모델 추론 실행 (session.prompt)
```

DeepWiki: <https://deepwiki.com/openclaw/openclaw>

# 계획 수립

## ❖ 모델 추론 실행 코드 전체 경로 – cont.

### [1] SystemPrompt 생성 코드

- export function **buildEmbeddedSystemPrompt**(params: {...}):  
: string { return **buildAgentSystemPrompt**({...})
- base system prompt 생성  
`const baseSystemPrompt = buildEmbeddedSystemPrompt ({...})`
- **before\_prompt\_build** hook 실행 (prompt injection hook 확인 및 반영)  
`const hookResult = await resolvePromptBuildHookResult ({...})`
- hook 결과와 base prompt 합성  
`const systemPromptOverride = composeSystemPromptWithHookContext (  
    {baseSystemPrompt, prependSystemContext, appendSystemContext})`
- **prependContext** (per-turn 동적 컨텍스트) 별도 처리  
(모델 컨텍스트를 관리하는 컨텍스트 엔진이 제공하는 컨텍스트)  
`prependSystemPromptAddition (activeSession, hookResult.prependContext);`